

Evaluation of Unsupervised Anomaly Detection in Structured API Logs

- A Comparative Evaluation with a Focus on API Endpoints

*Utvärdering av Oövervakad Anomalidetektering i Strukturerade
API-Loggar: en Jämförande Utvärdering med ett Fokus på API-
ändpunkter*

Gabriel Hult

Supervisor : Gurjot Singh
Examiner : Andrei Gurtov

External supervisor : Fredrik Seidl and Marcus Andersson

Upphovsrätt

Detta dokument hålls tillgängligt på Internet - eller dess framtida ersättare - under 25 år från publiceringsdatum under förutsättning att inga extraordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida <http://www.ep.liu.se/>.

Copyright

The publishers will keep this document online on the Internet - or its possible replacement - for a period of 25 years starting from the date of publication barring exceptional circumstances.

The online availability of the document implies permanent permission for anyone to read, to download, or to print out single copies for his/hers own use and to use it unchanged for non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional upon the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its www home page: <http://www.ep.liu.se/>.

Abstract

With large quantities of API logs being stored, it becomes difficult to manually inspect them and determine whether the requests are benign or anomalies, indicating incorrect access to an application or perhaps actions with malicious intent. Today, companies can rely on third-party penetration testers who occasionally attempt various techniques to find vulnerabilities in software applications. However, to be a self-sustainable company, implementing a system capable of detecting abnormal traffic which could be malicious would be beneficial. By doing so, attacks can be proactively prevented, mitigating risks faster than waiting for third parties to detect these issues. A potential solution is applying machine learning, specifically anomaly detection, which detects patterns that do not conform to normal standards. This thesis covers the process of having structured log data to find anomalies in the log data. Various unsupervised anomaly detection models were evaluated on their capabilities of detecting anomalies in API logs. These models were K-means, Gaussian Mixture Model, Isolation Forest and One-Class Support Vector Machine. The findings from the evaluation show that the Gaussian Mixture Model was the best baseline model, reaching a precision of 63%, a recall of 72%, resulting in an F1-score of 0.67, an AUC score of 0.76 and an accuracy of 0.71. By tuning the models, Isolation Forest performed the best with a precision of 67% and a recall of 80%, resulting in an F1-score of 0.73, an AUC score of 0.83 and an accuracy of 0.75. The pros and cons of each model are presented and discussed along with insights related to anomaly detection and its applicability in API log analysis and API security.

Acknowledgments

I want to express my deepest gratitude to everybody involved in the master thesis project. Firstly, I would like to thank my supervisor Marcus Andersson and client Fredrik Seidl as well as everyone else involved from Upsales for their technical expertise and support. They allowed me to perform this master thesis in the first place and I am very grateful for the opportunity. I would also like to thank my examiner Andrei Gurtov and supervisor Gurjot Singh for their consistent support and dedication throughout this master's thesis. This marks an end to my five-year-long journey of university studies, and I am greatly honoured to be able to share this journey with so many great people along the way.

- Gabriel Hult, June 2024

Contents

Abstract	iii
Acknowledgments	iv
Contents	v
List of Figures	vii
List of Tables	viii
List of Algorithms	x
List of Listings	xi
Acronyms	xii
1 Introduction	1
1.1 Motivation	1
1.2 Aim	2
1.3 Research questions	2
1.4 Contributions	2
1.5 Thesis Outline	2
1.6 Delimitations	3
2 Theory and Background	4
2.1 What are API logs?	4
2.2 Security	5
2.2.1 OWASP	5
2.2.2 API Security Risks	5
2.2.3 Intrusion Detection System	6
2.3 Anomaly Detection	7
2.3.1 Point Anomaly	7
2.3.2 Contextual Anomaly	7
2.3.3 Collective Anomaly	8
2.4 Unsupervised Learning	8
2.4.1 Clustering	9
2.4.2 Dimensionality Reduction	11
2.4.3 Other Unsupervised Approaches	12
2.5 Semi-Supervised Learning	13
2.6 Supervised Learning	13
2.7 Scikit learn	14
2.8 Z-Score Method	14
2.9 Summary of the theory	15

3	Related Work	16
3.1	Relevant literature	16
3.1.1	Selection process	16
3.1.2	Findings	18
3.2	Exploration of the Current Tools and Techniques	19
3.3	Techniques Tailored for API Traffic Anomaly Detection	20
3.4	Anomaly Detection Frameworks	22
3.5	Summary of Related Work	22
4	Method	24
4.1	Dataset	24
4.2	Anomaly Detection Process	26
4.2.1	Preparation Phase	27
4.2.2	Anomaly Detection Phase	29
4.2.3	Output Phase	31
4.2.4	Evaluation Phase	31
5	Result	36
5.1	Baseline Models	36
5.2	Tuning the Models	39
5.2.1	GMM	39
5.2.2	K-means	42
5.2.3	Isolation Forest	43
5.2.4	OCSVM	45
5.3	Tuned Models	47
6	Discussion	50
6.1	Results	50
6.1.1	Baseline Models	50
6.1.2	Tuning the Models	51
6.1.3	Tuned Models	51
6.2	Method	52
6.2.1	Dataset	52
6.2.2	Anomaly Detection Process and Reproducibility	52
6.2.3	Alternative Methods	54
6.3	Source Criticism	56
6.4	The work in a wider context	57
6.5	Final Discussions	57
7	Conclusions and Future Work	59
7.1	Research Questions	59
7.2	Future Work	60
	Bibliography	61
A	Appendix	67
A.1	Comparing the baseline clustering models	67
A.2	Comparing the tuned clustering models	68

List of Figures

2.1	Point anomalies at $x = 3$ and $x = 9$	7
2.2	Contextual anomaly in the range $5 \leq x \leq 7$	8
2.3	Illustration of a collective anomaly	8
3.1	Steps taken to identify literature	16
4.1	Proposed process of anomaly detection	26
4.2	Proposed preparation phase flow	27
4.3	WSSSE (Inertia) error graph with elbow point at $k = 4$	33
4.4	Example of a ROC-AUC curve	35
4.5	Example of a confusion matrix	35
5.1	Comparison of ROC curves for baseline models	38
5.2	Comparison of confusion matrices for baseline models	38
5.3	Silhouette using a different number of components	40
5.4	AIC using a different number of components	40
5.5	BIC using a different number of components	41
5.6	Silhouette scores using different numbers of clusters	42
5.7	Inertia (WSSSE) using different numbers of clusters	43
5.8	Comparison of ROC curves for tuned models	48
5.9	Comparison of confusion matrices for tuned models	49
6.1	Log parsing example	56

List of Tables

2.1	Information typically in API logs	4
2.2	Types of clustering	9
2.3	Summary of the theory	15
3.1	Inclusion and exclusion criteria for identified literature	17
3.2	Search string analysis	18
3.3	Summary of Current Tools and Techniques	22
3.4	Summary of the techniques tailored to API traffic anomaly detection	23
3.5	Summary of anomaly detection frameworks	23
4.1	Summary of the dataset	25
4.2	Description of the used CSIC log columns	25
5.1	PC specs used for evaluation	36
5.2	Performance metrics for baseline models	37
5.3	GMM metrics for different covariance types with 2 components	40
5.4	GMM tuning values with 50th percentile anomaly detection metrics	41
5.5	GMM metrics for different covariance types with 18 components	41
5.6	K-means tuning values with 50th percentile anomaly detection metrics	43
5.7	Isolation Forest performance across different contamination levels and estimators	44
5.8	PCA + OCSVM metrics for different kernels where $\nu = 0.5$ and $\gamma = 'scale'$	45
5.9	PCA + OCSVM parameter tuning results for RBF kernel	46
5.10	PCA + OCSVM metrics for different kernels where $\nu = 0.5$ and $\gamma = 'auto'$	46
5.11	Performance metrics for tuned models	48
A.1	Baseline clustering metrics and anomaly information for K-Means and GMM	67
A.2	Tuned clustering metrics and anomaly information for K-Means and GMM	68

List of Algorithms

1	K-means	10
2	Gaussian Mixture Model (GMM)	11
3	Principal Component Analysis (PCA)	12
4	Isolation Forest (IF)	12
5	One-Class Support Vector Machine (OCSVM)	13

List of Listings

4.1	Data cleaning	28
4.2	Feature engineering	28
4.3	Label encoding	28
4.4	Labels used after data processing task	29
4.5	Scaling data using PCA for clustering	29
4.6	Scaling data using PCA for OCSVM	29
4.7	Baseline models	30
4.8	Tuned models	31

Acronyms

AC Access Control

AIC Akaike Information Criterion

API Application Programming Interface

API-TAD API Traffic Anomaly Detection

ARI Adjusted Rand Index

AUC Area Under Curve

AWS Amazon Web Service

B2B Business to Business

BIC Bayesian Information Criterion

BGL BlueGene/L

CRLF Carriage Return Line Feed

CRM Customer Relationship Management

CSV Comma Separated Values

DBSCAN Density-Based Spatial Clustering of Applications with Noise

DDoS Distributed Denial of Service

DILAF Distributed Log Analysis Framework

DT Decision Tree

EC2 Elastic Compute Cloud

EM Exception-Maximisation

EMR Elastic MapReduce

FN False Negative

FP False Positive

FPR False Positive Rate

GMM Gaussian Mixture Model

HDFS Hadoop Distributed File System

HIDS Host Intrusion Detection System

IDS Intrusion Detection System
IM Invariants Mining
IF Isolation Forest
LANL Los Alamos National Laboratory
LG Log Clustering
LL Log-Likelihood
LOF Local Outlier Factor
LR Logistic Regression
MIME Multipurpose Internet Mail Extension
ML Machine Learning
NIDS Network Intrusion Detection System
NMI Normalised Mutual Information
OCSVM One-class Support Vector Machine
OWASP The Open Worldwide Application Security Project
PCA Principal Component Analysis
PoC Proof-of-Concept
RBF Radial Basis Function
RF RandomForest
ROC Receiver Operating Characteristic
S3 Simple Storage Service
SSRF Server Side Request Forgery
SVD Singular Value Decomposition
SVM Support Vector Machine
TN True Negative
TP True Positive
TPR True Positive Rate
WSSSE Within Set Sum of Squared Error
WCSS Within-Cluster Sum of Squares
XSS Cross-Site Scripting
XXE XML External Entity



1 Introduction

The following chapter presents an introduction to the master thesis. Firstly, motivation is given to understand the purpose. Then the aim of the thesis along with the research questions are presented. Finally, the chosen approach and current contributions are presented.

1.1 Motivation

The threat of security breaches in an Application Programming Interface (API) is a big concern and is at a point where The Open Worldwide Application Security Project (OWASP) ranks the ten most common API security risks every few years due to its critical nature [1]. Detecting vulnerable API endpoints is a challenging task and depending on the size of the service or product, the effort varies. Companies offering services and products via web applications have customers who can store and share sensitive company information which gives an incentive to have good and robust API security. However, should these companies be attacked through malicious activity against their system, it can lead to damage and in turn huge costs that can be challenging to recover from [2]. Currently, one approach is to use third-party penetration testers. These security experts can identify these vulnerabilities, rate their severity using OWASP-rating and provide a report on these vulnerabilities and how to mitigate them [3]. Although penetration testing is a good step towards improved security over the entire system [4], relying on third parties or manually inspecting these raw logs to identify exploitable API endpoints can be time-consuming, resource-heavy and error-prone. Getting an insight into what is considered abnormal activity towards API endpoints can be crucial for enhancing overall API security and preventing future security breaches.

Automating the inspection process can offer significant advantages by reducing human intervention and can lead to decreased costs. One increasingly interesting option is applying Machine Learning (ML) to identify abnormal activity towards API endpoints and their respective vulnerabilities. This option is anomaly detection which is identifying events that deviate from the expected outcome. Using anomaly detection for this can lead to an effective solution for companies to identify the attack surface of their API endpoints and prevent future security breaches. There are multiple frameworks available that supply different anomaly detection models, such as PyOD, Scikit-learn and Apache Spark [5, 6, 7]. What model is best suitable for detecting vulnerabilities via API logs depends on the system. Therefore, evaluat-

ing different models is a good starting point for future implementation of anomaly detection as part of improved API security [8].

1.2 Aim

This thesis aims to investigate and evaluate different approaches to detecting anomalies in API logs using unsupervised anomaly detection. This thesis will provide the process of using anomaly detection on API logs and which anomaly detection models can be applied to API log analysis. The effectiveness of the different unsupervised anomaly detection models, alongside their reliability in identifying anomalous requests towards API endpoints, will also be evaluated. Finally, the thesis will provide insights that can be gained from this evaluation for overall improved API security and what is required to start implementing an anomaly detection system in a real-life system.

1.3 Research questions

To evaluate different anomaly detection models in detecting abnormalities in API logs, the following research questions will be able to conclude this:

1. What is the process of implementing unsupervised anomaly detection in API logs, and what are the applicable models?
2. How effective, in terms of performance and time, are the different unsupervised anomaly detection models, specifically in detecting abnormal activity surrounding API endpoints?
3. What insights can be gained using unsupervised anomaly detection within API security, focusing on abnormal activities towards API endpoints?

1.4 Contributions

One of the main contributions this thesis will bring is how anomaly detection works for unsupervised approaches. Furthermore, how the chosen models and approaches performed concerning carefully selected evaluation metrics will provide insight into how well they perform on API logs. Lastly, general insight into the topic will be given such as the potential of unsupervised anomaly detection for API logs in real-life scenarios. These contributions lay a foundation for future work and future implementations of anomaly detection in an API system.

1.5 Thesis Outline

Chapter 2 introduces the theory and background needed to understand the rest of the thesis. Chapter 3 covers related work for the master thesis, where work with a similar goal is presented. In Chapter 4, the method for the thesis project is presented. Also, potential alternative methods are discussed, and the method itself is thoroughly explained. Chapter 5 presents the result showcasing what was achieved from the described method. Chapter 6 discusses the methods used, results retrieved and other relevant topics. Lastly, Chapter 7 concludes the master thesis and finalises the method, results, and discussions.

1.6 Delimitations

Conducted in collaboration with Upsales, a Business to Business (B2B) company providing Customer Relationship Management (CRM) solutions[9], this thesis focuses on their practical API security needs. So the evaluation will consider their desired results as much as possible.

Upsales will provide an OWASP-rated vulnerability report, where certain vulnerabilities discovered by a third-party company are found. The specifics of the vulnerability report will not be shared since that information is also confidential. However, this can be used as an indicator as to what vulnerabilities are interesting to investigate.

Another delimitation for the master thesis is the dataset which will be used. The initial plan was to use a real-world dataset provided by Upsales to detect anomalies. However, this proved not possible due to the nature of the dataset and the limited domain knowledge of the data acquired. Furthermore, the implications of the data containing sensitive information provide limited sharing in the thesis. Because of this, another web-based dataset will be used and is presented in Section 4.1. This dataset fulfils the focus of the master thesis but its limitation is discussed in Section 6.2.

Since evaluating different models is the focus of the master thesis, a complete implementation of an automated anomaly detection system for Upsales will not be performed. It would significantly increase the scope and be unachievable within the available time. Instead, a potential future implementation is investigated, which is the goal of Upsales and the author.



2 Theory and Background

This chapter will cover the necessary theory and background to understand the rest of the thesis. The chapter includes introductory theory to API logs and security theory to understand the goal of applying ML for this master's thesis. Furthermore, various ML methods and general knowledge of anomaly detection and the libraries used for the thesis will be presented.

2.1 What are API logs?

API logs are entries describing API calls performed by users. Typically in API logs, there exists critical information from the API call performed, as denoted in Table 2.1. [10]. This thesis will mainly focus on identifying anomalies towards the API endpoints. The data types can provide varying insights into the API call performed. For example, timestamps and status codes can be used to identify periods of failed requests which could potentially be anomalies.

Table 2.1: Information typically in API logs

API Log Information	Description
Timestamp	Indicating when the API call was made
Endpoint	What API endpoint was accessed
Request Parameters	Detailing the command requested
User Identification	Typically IDs or tokens used to identify users in a system
Response data	What the API returned to the caller
Status Codes	Indicating the success or failure of the request
IP Addresses and Device Information	Insight as to where and from what device the request originated

2.2 Security

The security-related theory is presented in this section to understand what context the ML will be applied to. Furthermore, knowing what API logs can reveal from a security perspective provides important knowledge to tackle the questions in this master thesis. Therefore, topics such as OWASP, API security risks and Intrusion Detection System (IDS) will be covered.

2.2.1 OWASP

OWASP is a nonprofit foundation that enhances software security. They are an open community that provides chapters, documents, forums, projects and tools free of use to anyone wanting to improve their application security [11].

Every few years, they publish an awareness document, including the most crucial security threats to web applications. For instance, the latest version of this top ten is from 2021, stating that *Broken Access Control* is the most critical vulnerability in web applications. These top tens are recommended to be adopted by companies looking to improve their security [12].

2.2.2 API Security Risks

APIs are critical components of modern software development, facilitating seamless interactions between different systems and applications. However, their openness and accessibility also make them susceptible to various security vulnerabilities. This section delves into common API-security risks where some are mentioned by OWASP. Various API-security risks often relate to web-security risks. However, it is important to realise that they differ slightly. APIs in general give access to clients through Access Control (AC). When making requests to an API, a lot more information is exposed to the end user compared to a simple web request which only exposes the HTTP URL, such as the HTTP header, queries and other parameters. Due to this, the attack surface has a risk of increasing, which could lead to an increased number of attempted attacks towards the API [13].

As mentioned in Section 2.2.1, the nonprofit foundation OWASP plays a crucial role in enhancing software security through its comprehensive resources and guidelines, including its periodically updated awareness document detailing critical security threats to web applications. The vulnerabilities outlined below align with the concerns identified by OWASP, underscoring the relevance of their work to the specific challenges faced by APIs. The vulnerabilities introduced in this section reflect the security risks highlighted by OWASP, particularly in their API-security risks list. By focusing on these vulnerabilities, this work aligns with OWASP guidelines and aims to contribute to the broader effort of securing APIs against modern threats.

Script Injections

Script injection is a vulnerability where an attacker sends malicious scripts through an API request, which the system then mistakenly executes or processes as a valid parameter. This can lead to unauthorised actions being performed, such as data theft, account compromise, or the execution of malicious code on the server or client-side applications.

SQL Injections

SQL injection is a prevalent vulnerability occurring when an attacker manipulates a standard SQL query through user input exploitation. By crafting input data that alters the query's structure, an attacker can gain unauthorised access to the database, manipulate or steal sensitive data, and perform other malicious activities.

Buffer Overflows

Buffer overflows exploit a vulnerability where an application or API fails to check the boundary of a buffer (memory storage), allowing an attacker to overload this buffer with excessive data. This can cause system crashes, data corruption, or worse, provide attackers with the means to execute arbitrary code, potentially taking over the affected system.

Distributed Denial of Service (DDoS) Attacks

Distributed Denial of Service (DDoS) attacks target APIs by overwhelming the target with a flood of traffic, significantly exceeding the server's capacity to handle requests. This can render the API and associated services unavailable, disrupting operations and access for legitimate users.

Login Attacks

Login attacks exploit vulnerabilities in authentication and authorisation mechanisms. Techniques such as fuzzing—sending large volumes of random data to discover security, loopholes and credential stuffing—using stolen username and password combinations to gain unauthorised access—are common. These attacks aim to bypass security measures and access restricted areas or sensitive information within an application [13].

Some of the API-security risks brought up by OWASP can be classified as login attacks. *Broken Authentication* and *Broken Function Level Authorisation* can be considered as login attacks since the attacker might use stolen tokens or credentials which leads to unauthorised access [13, 1].

Server-Side Request Forgery (SSRF)

Server Side Request Forgery (SSRF) represents a critical vulnerability where an attacker can force the server to execute unauthorised requests to internal or external resources. This can lead to the exposure of sensitive information, server configurations, or unauthorised access to internal services. By manipulating the server to make requests on their behalf, attackers exploit SSRF vulnerabilities to bypass security measures and access restricted data [14].

2.2.3 Intrusion Detection System

An intrusion is an unauthorised action of compromising the security of a computer or a network in terms of confidentiality, integrity and availability [15]. An IDS is a software tool to detect unauthorised network or computer system access. Its capabilities include detecting malicious traffic, including attacks against services deemed vulnerable. Host-based attacks such as privilege escalation, unauthorised logins and access to sensitive files are also something that an IDS can detect. This also applies to network-based attacks such as DDoS attacks and spoofing. The motivation behind using an IDS is to complement a static monitoring system, the firewall, with a dynamic monitoring system. The IDS analyses the packets received from a user by checking whether there are any rule violations, which are detected using a pattern recognition algorithm. When a violation is detected, it alerts system administrators that something suspicious has occurred and needs attention [16].

There is said to exist a threat model where different types of threats can be divided into three separate classes, *external penetrations*, *internal penetrations* and *misfeasance*. External penetrations cover intrusions that unauthorised computer system users carry out. Furthermore, internal penetrations cover where users who are authorised but not the specific data being compromised. Finally, misfeasance covers the misuse of authorised access both to the system and to its data [15, 16]

2.3 Anomaly Detection

Anomaly detection is examining data points and identifying suspicious occurrences, commonly referred to as *outliers* due to their deviation from the established pattern of behaviour. Anomaly detection is a fundamental technique for various applications, especially real-time applications where security applications are one of them. There are multiple techniques applicable to anomaly detection such as classification, regression and clustering [17]. The different approaches are most commonly supervised or unsupervised. However, semi-supervised is also a viable approach. The techniques mentioned above can be matched to these approaches, clustering is considered an unsupervised approach for instance while classification and regression are regarded as supervised techniques. Various types of anomalies in logs can exist such as *point anomaly* and *contextual anomaly*.

2.3.1 Point Anomaly

A point anomaly refers to a particular independent data instance deviating from the normal pattern of the provided data [18, 15, 19]. An intuitive example of a point anomaly is when a user tries to access restricted sections of a computer system [18] or when someone e.g. usually uses three resources at once instead of nine resources. Point anomalies are the most commonly assumed types of anomalies and are what many anomaly detection systems reference. However, point anomalies are ineffective when attempting to find anomalies in a particular context. In Figure 2.1 [18], two small groups and two point anomalies are illustrated to showcase how they could be visually represented.

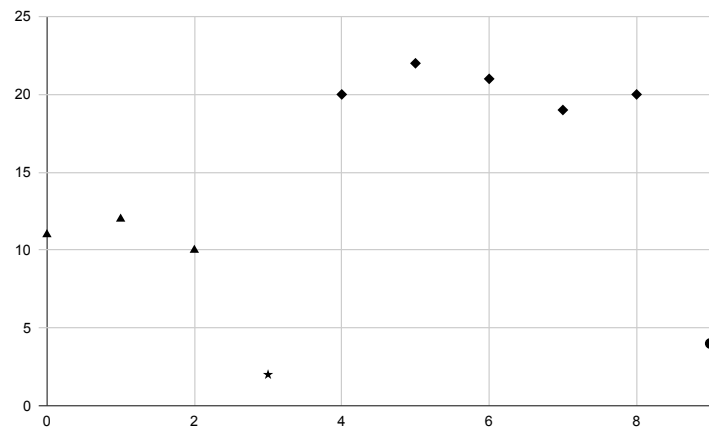
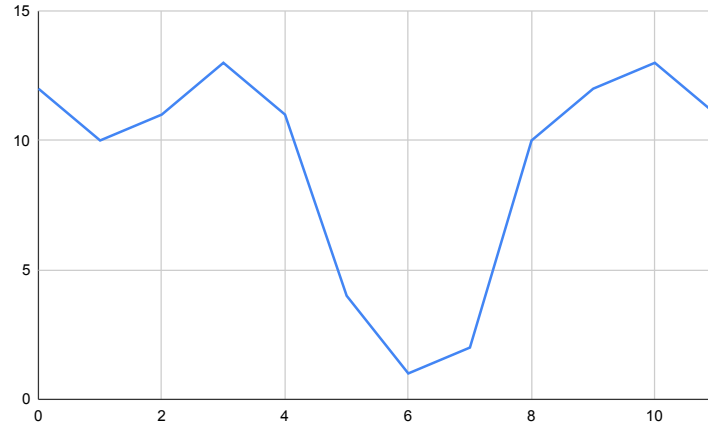


Figure 2.1: Point anomalies at $x = 3$ and $x = 9$

2.3.2 Contextual Anomaly

In some situations, an independent data point is not enough to detect anomalies. In these situations, the behaviour is defined within a particular context [19]. The data instance of the temporal context involves multiple attributes from a given dataset, such as location-relevant or time-relevant. An example of a time-relevant anomaly could be when someone logs in to their business system outside of working hours when the user usually only logs in during those hours [18]. Logging in from another location, such as another city, can also be seen as a contextual anomaly in a location-relevant context. Figure 2.2 [18] presents a visual example of a contextual anomaly.

Figure 2.2: Contextual anomaly in the range $5 \leq x \leq 7$

2.3.3 Collective Anomaly

Instead of a single data instance being anomalous, a collective anomaly refers to a group of data instances behaving anomalously concerning the entire dataset [19]. A single instance in the collective anomaly group does not have to be an anomaly on its own [18]. An example could be that someone is measuring their pulse and for the entire measurement time, it is detected that the pulse increased and stay increased for a while before returning to normal. That group of higher pulse values could be considered a collective anomaly. The example is presented in Figure 2.3 [18], where *Process 3* is the collective anomaly concerning the other processes in the dataset.

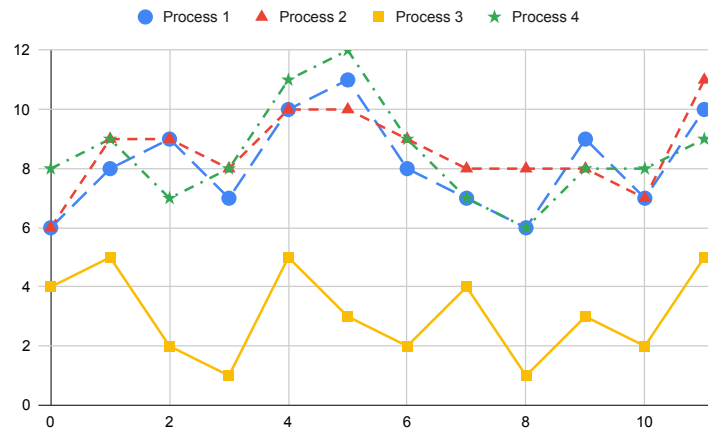


Figure 2.3: Illustration of a collective anomaly

2.4 Unsupervised Learning

Unsupervised learning is a category of ML which trains on data without human supervision or prior knowledge. Unsupervised models do not use labelled data for discovering patterns without human guidance or instruction. There are typically three unsupervised ML methods that are used, those are *clustering*, *association rules* and *dimensionality reduction*. For this master

thesis, clustering and dimensionality reduction will be relevant and therefore presented in more detail.

2.4.1 Clustering

Clustering is a technique for exploring and dividing unlabelled data into smaller groups named clusters, based on their similarities and differences. The cluster algorithms split the analysed data into groups by identifying the similarities in the pattern of the non-categorised data. Clustering is one of the more commonly used unsupervised approaches and multiple clustering approaches are available. Table 2.2 shows a subset of available clustering approaches.

Table 2.2: Types of clustering

Clustering Type	Description
Exclusive Clustering (Hard Clustering)	Data are grouped so a single data point can only exist in one cluster.
Hierarchical Clustering	Data are divided into distinct clusters based on their similarities, which are repeatedly merged and organised based on hierarchical relationships.
Overlapping Clustering (Soft Clustering)	Data is grouped such that a single data point can exist in several clusters, using a degree of membership to each cluster.
Probabilistic Clustering	Groups data points into clusters based on their probability of belonging to a specific cluster.

K-means

K-means is one of the most common models used within the unsupervised learning paradigm. It is a hard clustering approach where the goal is to locate *centroids* which can be seen as a reference point where the data of a set can be compared. Centroids can in simpler terms be seen as centres of mass. Only one centroid can exist per cluster. Algorithm 1 presents the flow of using K-means where convergence is reached when the two consequent iterations are identical.

Algorithm 1 K-means

```

1: procedure KMEANS( $k, D$ )
2:   Initialise  $k$  centroids randomly from dataset  $D$ 
3:   repeat
4:     for each data point  $x$  in  $D$  do
5:       Assign  $x$  to the nearest centroid using either Euclidean or cosine distance
6:     end for
7:     for each cluster do
8:       Update the centroid to be the mean of its data points  $d$  (centroid  $\leftarrow \frac{1}{|d|} \sum_{x \in d} x$ )
9:     end for
10:    Calculate the variation of the new centroids  $C$  ( $WCSS(C_i) \leftarrow \sum_{x \in C_i} \|x - \mu_i\|^2$ )
11:  until Two cluster assignments and centroids are the same ( $C^{(t)} = C^{(t+1)} \quad \forall t \geq T$  and  $\|\mu^{(t+1)} - \mu^{(t)}\| < \text{tol} \quad \forall t \geq T$ )
12:  Output: Set of clusters with centroids
13: end procedure

```

Gaussian Mixture Model (GMM)

For probabilistic clustering, Gaussian Mixture Model (GMM) is available. GMM is an Expectation-Maximisation (EM) type of model, in other words, an algorithm performing maximum likelihood on latent variables. Latent variables are either directly observed measured or indirectly inferred through mathematical models. The goal of GMM is to find a model's maximum likelihood by optimising its parameters. It is assumed that all data points are part of a mixture of normal distributions for their parameters are unknown. GMM performs soft classification, meaning that it does not assign a probability of belonging to a set of clusters, but can belong to several simultaneously. Clusters created by GMM are assumed to follow a Gaussian shape, determined by two parameters: the mean (μ) and the standard deviation (σ or Σ). These two parameters are used for GMM to be maximised.

Calculating the EM model GMM is divided into two major steps, step E and step M. Step E (Algorithm 2.1) for calculating the probability that a point (x_i) belongs to each cluster, c_k . Step M (Algorithm 2.2 and Algorithm 2.3) for updating the values μ , Σ , and Π (representing the point density of a distribution). Both steps will be performed iteratively, optimising the parameters and maximising the associated likelihood function.

$$r_{ic} = \frac{\text{Prob. of } x_i \text{ belonging to } c}{\sum \text{Prob. of } x_i \text{ belonging to } c_1, c_2, \dots, c_k} \quad (3) \quad (2.1)$$

$$\Pi = \frac{\text{Number of points assigned to a cluster}}{\text{Total number of points}} \quad (4) \quad (2.2)$$

$$\mu = \frac{\sum r_{ic} x_i}{\text{Number of points allocated to a cluster}} \quad (5) \quad (2.3)$$

Algorithm 2 Gaussian Mixture Model (GMM)

```

1: procedure GMM( $X, k, \text{max\_iter}$ )
2:   Initialize  $k$  Gaussian distributions with parameters  $\theta = \{\mu, \Sigma, \pi\}$ 
3:   for iteration = 1 to max_iter do
4:     /* E-Step */
5:     for each data point  $x \in X$  do
6:       Calculate responsibility  $\gamma(z_{nk})$  for each cluster  $k$ 
7:     end for
8:     /* M-Step */
9:     for each cluster  $k$  do
10:      Update parameters  $\mu_k, \Sigma_k, \pi_k$  using the responsibilities
11:    end for
12:  end for
13:  /* Compute the log-likelihood */
14:  Compute  $L(\theta) = \sum_{i=1}^N \log \sum_{k=1}^K \pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)$ 
15:  until convergence of  $L(\theta)$ 
16:  return model parameters  $\theta$  and responsibilities  $\gamma$ 
17: end procedure

```

2.4.2 Dimensionality Reduction

Dimensionality reduction is another unsupervised learning approach where the number of features is reduced from the dataset. Even though more data is seen as better regarding ML, it can also make it more challenging to visualise the data or compute it due to the high complexity. This approach extracts the important features, reducing the unnecessary or random features in the dataset. This approach utilises algorithms such as Principal Component Analysis (PCA) and Truncated Singular Value Decomposition (SVD) to reduce the number of data inputs without compromising the original data [20].

Principal Component Analysis (PCA)

PCAs is a statistical linear approach to e.g. visualise data using linear combinations of p random variables $(A_1, A_2, \dots, A_p)$. There are three important properties required for PCA.

1. The principal components are uncorrelated
2. The first principal component has the highest covariance, the second principal component has the second highest covariance etc.
3. The total variation of all the components is equal to the total variation in the original variables $(A_1, A_2, \dots, A_p)$

When these properties are met, one can utilise PCA to lower the number of dimensions in the data, following Algorithm 3 [19, 21, 22]. The number of principal components used matters since the goal is to reduce the number of dimensions as much as possible but minimise the lost variance in the data.

Algorithm 3 Principal Component Analysis (PCA)

```

1: procedure PCA( $X, d$ )
2:   Standardise the dataset  $X$  to have mean 0 and variance 1
3:   Compute the covariance matrix  $\Sigma$  of the standardised dataset  $X$ 
4:   Compute the eigenvalues  $\lambda_i$  and eigenvectors  $v_i$  of  $\Sigma$ 
5:   Sort eigenvalues and corresponding eigenvectors in decreasing order
6:   Select the top  $d$  eigenvectors  $v_1, v_2, \dots, v_d$  based on sorted eigenvalues
7:   Form the projection matrix  $W$  from the selected eigenvectors
8:   Transform the original dataset  $X$  using  $W$  to get the reduced dataset  $Y = XW$ 
9:   return Reduced dataset  $Y$ 
10: end procedure

```

2.4.3 Other Unsupervised Approaches

Besides clustering and dimensionality reduction, there are other unsupervised approaches in the world of anomaly detection, such as Isolation Forest (IF) and One-class Support Vector Machine (OCSVM), these will be described in this subsection.

Isolation Forest

IF is an ensemble-based, unsupervised outlier detection method that utilises binary isolation trees to identify anomalies within a dataset. This technique leverages the principle that anomalies are 'few and different', making them easier to isolate compared to normal points. In IF, a forest of binary trees is constructed by randomly sampling subsets of the dataset, with each tree grown to a specified height limit. The key to IF's effectiveness lies in its unique approach to anomaly scoring: for each data point, the path length from the root to the leaf node is calculated across all trees in the forest. Shorter paths suggest a higher likelihood of the data point being an anomaly.

The construction of the isolation trees within IF is governed by a specific algorithm, detailed in Algorithm 4. Here, X denotes the input dataset, while t and ψ represent the number of trees in the forest and the subsampling size used for each tree, respectively. The height limit for tree growth, calculated based on ψ , ensures that the isolation process is effective, facilitating the identification of anomalies through their isolation levels as quantified by the path lengths in the isolation trees [23].

Algorithm 4 Isolation Forest (IF)

```

1: procedure ISOLATIONFOREST( $X, t, \psi$ )
2:   Initialize forest  $F$  as an empty list.
3:   Determine the height limit  $l$  as  $\text{ceil}(\log_2 \psi)$ .
4:   for  $i = 1$  to  $t$  do
5:     Randomly sample  $\psi$  instances from  $X$  to form a subset  $X'$ .
6:     Grow an isolation tree  $T_i$  from  $X'$  up to the height limit  $l$ .
7:     Add  $T_i$  to the forest  $F$ .
8:   end for
9:   for each instance  $x \in X$  do
10:    Compute  $h(x)$ , the average path length from the root node to  $x$  over all trees in  $F$ .
11:   end for
12:   Compute anomaly scores  $s(x, n) = 2^{-\frac{h(x)}{c(n)}}$  for all  $x \in X$ , where  $c(n)$  is the average path length in a Binary Search Tree (BST) of size  $n$ .
13:   return Anomaly scores  $s(x, n)$  for all instances  $x$  in  $X$ .
14: end procedure

```

OCSVM

OCSVM is a specialised version of the Support Vector Machine (SVM) used for anomaly detection, particularly suited for datasets where the majority of data points are considered 'normal', and the goal is to detect outliers or anomalies. Unlike traditional SVM, which is used for classification or regression tasks involving two or more classes, OCSVM focuses on distinguishing normal data points from outliers by learning a decision function that isolates the normal data in the feature space.

OCSVM works by mapping input data points into a high-dimensional feature space using a kernel function and then finding the maximal margin hyperplane that separates the origin in this feature space from the mapped normal data points. Data points located on the side of the hyperplane closer to the origin are considered outliers, while those on the other side are considered normal. Using kernel function allows OCSVM to capture complex relationships in the data and effectively isolate outliers, even in cases where the boundary between normal data and anomalies is not linearly separable in the original feature space [24, 25].

The effectiveness of OCSVM depends on the choice of kernel and its parameters, as well as the ν parameter (ν), which controls the trade-off between the fraction of outliers and the decision function's margin. A small value of ν tightens the decision boundary around the normal data, potentially increasing the sensitivity to outliers. In contrast, a larger ν allows for more flexibility but may include some outliers within the normal region.

Algorithm 5 One-Class Support Vector Machine (OCSVM)

```

1: procedure OCSVM( $X, \nu$ , kernel, kernel_params)
2:   Initialize OCSVM with specified  $\nu$ , kernel, and kernel parameters.
3:   Fit OCSVM model to the dataset  $X$  assuming it represents the "normal" data.
4:   The model learns a decision function that isolates the normal data from the origin in
      the feature space.
5:   for each instance  $x \in X$  do
6:     Compute the decision function  $f(x)$ .
7:     if  $f(x) < 0$  then
8:        $x$  is an outlier.
9:     else
10:       $x$  is considered normal.
11:    end if
12:  end for
13:  return decision scores or labels for each instance in  $X$ .
14: end procedure

```

2.5 Semi-Supervised Learning

Semi-supervised learning is also one category of ML that can be seen as a combination of unsupervised and supervised learning. It uses a small amount of labelled data and a large amount of unlabelled data. The goal of semi-supervised learning is similar to supervised learning in the sense that it is training on predicting the categorical label or output, based on the input data. However, it is also trained on unlabelled data, not just labelled data [26].

2.6 Supervised Learning

Supervised learning is another category of ML which, unlike unsupervised learning, uses labelled datasets to train various algorithms for predicting outcomes and recognising patterns. Two commonly used supervised learning approaches are *classification* and *regression*. Classification is a group of algorithms used to group data based on their categorical label or output, based on the input data. It is typically used when it exists two or more classes to categorise.

On the other hand, regression algorithms are used to predict real or continuous values, where the different algorithms detect relationships between two or more variables [27].

2.7 Scikit learn

Scikit-learn (also known as sklearn) is a module for Python containing a wide area of state-of-the-art ML algorithms to be used in medium-sized supervised and unsupervised problems. Scikit-learn aims to bring ML to non-specialists using a general-purpose high-level language, like Python. The module has emphasised e.g. ease-of-use and performance and is distributed under a BSD license, encouraging its use in academia and the industry.

Some of Scikit-learn's core features involve supervised and unsupervised learning approaches for solving or analysing general problems. For supervised learning, algorithms such as SVM and RandomForest (RF) are available through the module. For unsupervised learning, Scikit-learn supports e.g. K-means and PCA. Other features such as tools for feature extraction and metrics are also available using Scikit-learn. Another important feature available is data scaling, specifically StandardScaler, which utilises Z-score normalisation mentioned in Section 2.8.

Scikit-learn's consistent API design makes it highly accessible to beginners while maintaining enough powerful actions for experts. Its integration with other libraries in Python such as *NumPy* and *SciPy* for mathematical operations, *pandas* for data manipulation and *matplotlib* for data visualisation makes it a very versatile module for various data science projects [6].

2.8 Z-Score Method

The Z-score method is a common statistical approach which measures the distance of a data point from the mean of a dataset in terms of standard deviations. The formula for calculating the Z-score is described in Eq. (2.4), where x is the specific datapoint, μ is the mean and σ is the standard deviation of the entire distribution [28].

$$z = \frac{x - \mu}{\sigma} \quad (2.4)$$

Using the Z-score for anomaly detection can be based on that if a data point has a Z-score of $\pm n$, where n is the chosen sensitivity, the data point can be considered unusual. However, the Z-score depends on whether the dataset used follows the standard distribution model [29].

2.9 Summary of the theory

Table 2.3 summarises the theory presented in Chapter 2, briefly explaining each section and subsection containing the theory for the master thesis.

Table 2.3: Summary of the theory

Section	Summary
2.1	API logs are entries describing calls made to an API, containing critical information from the request such as endpoint, timestamp and response data.
2.2.1	OWASP is a nonprofit foundation that focuses on enhancing the security of various software. Every few years, they release documentation and lists of current exploits which are considered large vulnerabilities for different software systems such as API and other web applications.
2.2.2	The API has various vulnerabilities which can result in attacks such as script injection, SQL injection, login attacks, DDoS attacks and other application and data attacks. Some of these fall in line with the OWASP top 10 API-security risks.
2.2.3	An IDS is a system for detecting unauthorised actions compromising the security of a computer system. There exist several types of breaches such as host-based attacks and network-based attacks, and users can have an IDS for each, a Network Intrusion Detection System (NIDS) and Host Intrusion Detection System (HIDS).
2.3	The process of examining and identifying data points seen as abnormal from the normal pattern
2.3.1	A point anomaly is an independent data instance deviating from the normal pattern
2.3.2	A contextual anomaly is when anomalies are defined in specific contexts, such as time-relevant or location-relevant, instead of only being treated with regards to the entire dataset
2.3.3	A collective anomaly is a group of data points behaving anomalously concerning the given dataset
2.4	Unsupervised learning is a category in ML training models on unlabelled datasets, i.e. data that does not tell what the outcome should be. Unsupervised learning has the objective of dividing the data into categories based on score. Different types of unsupervised learning are clustering and dimensionality reduction.
2.5	Semi-supervised learning is a ML category seen as a mix of unsupervised and supervised learning. The data is partially labelled while the remaining dataset stays unlabelled. The goal is to train the models to predict the categorical label or output, as in supervised learning.
2.6	Supervised learning is a category in ML utilising labelled datasets to train models to predict an outcome by recognising patterns. The objective is to predict the correct categorical label or output. Different types of supervised learning are linear regression and classification.
2.7	Scikit-learn is a module available in Python which includes a wide amount of ML algorithms used for supervised and unsupervised problems. Scikit-learn offers core classification, regression and clustering algorithms and more.
2.8	Z-score is a statistical approach measuring the distance of a data point from the mean of a dataset in terms of standard deviations.

3 Related Work

Relevant literature and related works are presented in this chapter to get a baseline of knowledge about previous attempts, and what those evaluations have led to. Previous evaluations have been made on models and frameworks and how they perform regarding log analysis. Of all the related work available, only a few of them focused solely on API traffic and corresponding logs at their respective endpoints. Therefore, the related work will cover how anomaly detection methods were used for system logs in general and API logs, and how they were used and evaluated.

3.1 Relevant literature

A literature study has been conducted to acquire a thorough knowledge base of implementing anomaly detection for logs and what approaches could be appropriate for API log analysis or similar log analysis purposes. The literature study will aid with awareness of current challenges to prevent pitfalls in the evaluation process. This follows what was also performed by Sowmya et al. [30] and Baye et al. [8] in their works. Furthermore, the literature base is used to help formulate a reasonable methodology for the evaluation process of the master thesis as well as to know what models have been used and what their results were. The literature study process consists of the steps illustrated in Figure 3.1. Databases used include *IEEE Xplore*, *DiVA LiU* and *ScienceDirect*. All used databases are listed in Table 3.2

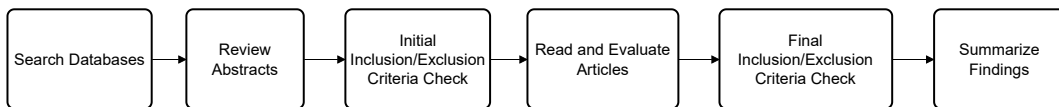


Figure 3.1: Steps taken to identify literature

3.1.1 Selection process

To select relevant literature, a process is in place to filter papers which will not contribute to the result in a significant manner. The selection process is performed in three steps, which are as follows:

1. **Identification of search keywords** - Keywords and synonyms to these keywords that could assist in finding relevant sources are identified. Keywords *anomaly detection*, *assessment*, *unsupervised*, *api*, *traffic*, *security*, *logs*, *frameworks*, *models*, *techniques* and *statistical* were used in this matter.
2. **Construction of search strings** - Using the keywords as mentioned earlier, independent search strings can be constructed to find relevant literature using Boolean Operators. The term *AND* was used to find sources where both keywords were present in the paper, but not explicitly next to each other. Furthermore, the term *OR* was used to find sources where either one of the words or both were present in the papers found. Finally, *NOT* did not have any use so was not utilised in this literature study [31].
3. **Inclusion/Exclusion criteria** - Sources are validated and filtered according to the inclusion and exclusion criteria described in Table 3.1. This is made in two sessions. For the first session, the initial inclusion/exclusion criteria check, the abstract was read and the text was skimmed to get a basic understanding of the paper. For the second session, the final inclusion/exclusion criteria check, the paper was read in more detail to understand the text and whether it was relevant to this thesis. Should some exclusion criteria be met, it is still possible to include it if it contains a lot of other inclusion criteria.

Table 3.1: Inclusion and exclusion criteria for identified literature

Inclusion Criteria	Rationale
• Papers that assess the process of anomaly detection. • Papers that can justify why the process works. • Papers describing anomaly detection approaches to API/network security, preferably unsupervised or statistical. • Papers describing the evaluation process. • Papers published in trustworthy databases. • Papers published within the last 10 years, if possible.	• To understand the effectiveness of anomaly detection in API logs. • To ensure a theoretical and practical foundation. • To focus on relevant anomaly detection approaches for API/network security. • To assess the methodologies used in evaluating anomaly detection. • To ensure the reliability and credibility of the sources. • To focus on recent developments and practices.
Exclusion Criteria	Rationale
• Non-English papers. • Lacking justification on why the process works. • Papers lacking explanation of evaluation metrics. • Papers not including unsupervised or statistical approaches. • Papers with a focus on deep learning • Duplicates	• Language barrier and potential difficulty in comprehension. • Ensuring a strong theoretical foundation for the evaluated anomaly detection process. • To ensure a comprehensive assessment of anomaly detection approaches. • To focus on unsupervised and statistical approaches for API/network security. • This master thesis has no intent to include deep learning. • To avoid redundancy in collecting papers.

3.1.2 Findings

By using the selection process for the master thesis, 84 papers were initially identified including a snowballing search, where 41 remained after the initial inclusion/exclusion check. After reading further in the remaining papers, the final inclusion/exclusion check was made to filter papers based on the entire content, instead of only the abstract. Finally, this led to the summarising phase where the 23 included papers were utilised. In Table 3.2, the search string and their corresponding results are displayed to show what papers were taken from one of the keyword search strings or the snowballing search. The papers where the snowballing search was performed are from the papers chosen from the initial screening. This ensures that even though papers used for snowballing searches are filtered out, papers which technically could be more relevant can still be identified.

Table 3.2: Search string analysis

Keyword Search Strings	Identified	Screening	Included	Selected Papers	Source
('unsupervised' OR 'statistical') AND 'anomaly detection' AND (('api' AND ('traffic' OR 'security' OR 'logs')) OR 'techniques' OR 'models' OR 'frameworks')	24	14	6	[16] [32] [21] [33] [34] [35]	ScienceDirect IEEE MDPI IEEE ACM IEEE
('assessment' OR 'measuring' OR 'evaluating') AND 'anomaly detection' AND 'process' AND ('api' OR ('api' AND ('traffic' OR 'security')) OR 'security' OR ('security' AND 'logs'))	18	9	5	[36] [37] [38] [39] [40]	Swinburne Oulu IEEE IEEE IEEE
('anomaly detection' OR 'outlier detection') AND (('api' AND ('logs' OR 'traffic')) OR 'models' OR 'frameworks' OR 'techniques'	23	6	2	[30] [41]	IEEE Wiley
Snowballing search	19	12	10	[19] [42] [43] [44] [45] [46] [47] [48] [49] [50]	ScienceDirect IEEE ACM IEEE IEEE ScienceDirect ScienceDirect Wiley ACM ArXiv
Total	84	41	23		

3.2 Exploration of the Current Tools and Techniques

When choosing what anomaly detection models to evaluate to narrow the scope, knowing what to choose and why can become tricky. As observed by He et al. [33], there exists a gap between the research of anomaly detection in academia and the use of it practically in the industry. Therefore, He et al. investigated how six different state-of-the-art anomaly detection methods performed in system log analysis, using two publicly available datasets (Hadoop Distributed File System (HDFS) and BlueGene/L (BGL)) totalling 15,923,592 log messages and 365,298 anomaly instances. He et al. provided the overall procedure they followed for their anomaly detection.

First, they performed log collection which consisted of the timestamp and what action was performed. Continuing, they performed log parsing to extract a group of events containing the event template (constant part) and specific parameters (variable part). After that, feature extraction was performed to encode the logs and numerical feature vectors so that ML models could be applied. Finally, the logs were given to the model to be trained, validated and tested. For this evaluation, three supervised approaches and three unsupervised approaches were used. The chosen unsupervised approaches were Log Clustering (LG), PCA and Invariants Mining (IM), while the chosen supervised approaches were Logistic Regression (LR), Decision Tree (DT) and SVM.

The performed evaluation was based on precision, recall and efficiency. Furthermore, 80% of the data was used as training data, while the remaining 20% was used as testing data. The results showed that when testing the supervised approaches, SVM had the best balance of precision and recall (F1-score) while the decision tree approach had the worst balance. Meanwhile, testing the unsupervised approaches showed that invariant mining was a clear winner with an F1-score of 0.91 while the others were around 0.56.

In conclusion, supervised approaches performed better for the most part, with the unsupervised approach invariant mining showing good potential. He et al. state that more methods have been evaluated such as *frequent sequence mining*, *finite state machine*, *formal concept analysis* and *information retrieval* by referring to multiple sources showing this evaluation. They also believe more approaches are being released because of the increasing importance of log analysis. This provides the potential for further evaluation surrounding anomaly detection in logs. Even though supervised methods will not be used in this master thesis, it provides insight into how the performance of unsupervised approaches compares to supervised ones.

LiU students Franzén et al. [51] investigated current tools and techniques and how anomaly detection could be applied for automatic log analysis. One of the evaluated tools was *ELK stack* which is a collection of open-source components, such as *Elasticsearch*, *Logstash* and *Kibana* [52]. They also implemented a Proof-of-Concept (PoC) where they evaluated K-means in combination with PCA on an identical dataset as He et al. [33], which was the HDFS dataset. This dataset version contained 11,175,629 log messages with 16,838 labelled anomalies. They used a log parsing application similar to He et al. to parse the raw logs in possession. The authors proceeded with standardising and normalising the features using StandardScaler provided by Scikit Learn. They also utilised dimensionality reduction, specifically PCA to reduce the number of dimensions. A warning was placed to make the readers understand that PCA can potentially negatively impact the precision and recall of the models if enough numerical data is lost when compressed. To evaluate, the authors used precision, recall and F1-score on the concluded outliers using K-means. The concluded outliers were decided based on a percentile threshold where if an event vector had a greater distance from its centroid than the percentile, the event would be classed as an outlier.

Their results reached a precision of 73,7% and a recall of 100%, showing that anomaly detection was useful for detecting unusual activity. However, detecting and protecting against unknown threats is a challenging, potentially even impossible task. The authors also took

notice of a potential problem with anomaly detection, the high false-positive rate. They concluded that this could make it a necessity to perform further analysis.

Kumari et al. [32] attempted to use the unsupervised K-means clustering approach to detect network traffic anomalies. On top of the use of K-means, *Apache Spark* was used as the core framework for the experiment. As of the writing of the paper, the framework was fairly new. It is used for iterative ML analytics and at the time of the paper's publication, K-means was supported, hence its use. Specifically, the *MLlib*, a ML library for Spark, was used. The library and its rich number of features have been utilised in numerous studies, including the research conducted by Kumari et al. [32] but also by Astekin et al. [38, 44, 48]

The experiment ran twice where the initial run did not choose k while in the second run, different values for k were chosen. The way the K-means algorithm operated was such that it checked what connections tried to connect to the network. If a connection was not similar to past benign connections, it was determined as an outlier. When the different runs were performed, a visualisation of the data was done to better analyse the results and to draw early conclusions about the impact some data made. Continuing, the clustering was performed with normalisation. Even though this would move all the data points relatively the same distance, a richer structure was obtained by observing the visualisation. It was noticed that one larger cluster dominated the other smaller clusters.

To be able to analyse further, clustering with categorical variables was performed. This was done to better separate the data in the clusters. This proved Kumari et al. to be optimum clusters. Finally, they used entropy as a way to check the quality of the clusters. A good cluster was determined to contain network activities of similar types while a bad cluster contained activities of dissimilar types. As a result of this procedure, it could be determined that K-means could be used as a clustering technique to detect anomalies. Together with Spark, this could be used in real-time to detect anomalies which could be network intrusions. In other words, it could be a potential deployable network intrusion detection system for detecting anomalies in the network traffic. This study suggests that the unsupervised approach K-means can be used to detect anomalies in network traffic, which is not far-fetched from API data since network traffic is usually stored in log files similar to API activity.

3.3 Techniques Tailored for API Traffic Anomaly Detection

Sowmya et al. [30] used a technique for anomaly detection regarding API traffic, named API Traffic Anomaly Detection (API-TAD). It was evaluated on a microservice architecture. Similar to Upsales, which uses Amazon Web Service (AWS) as its API service. The authors state that a single mismanaged API could prove to be a backdoor for attackers, compromising the company's classified data. Sowmya et al. claim that APIs can be an important source for tracking abnormal behaviour and that there have been few works surrounding anomaly detection on API traffic, such as the SVM approach made by Baye et al. [8].

The dataset used by Sowmya et al. was from a Banking Application which included attributes such as *timestamp*, *client IP address*, *API endpoint*, *response data size in bytes*, *status code* and *request type*. The dataset was stored on a Firebase Firestore database, a cloud service for database storage [53].

The anomaly detection that was performed and evaluated was made on two layers. The first layer was a generalised layer which was used to detect large-scale attacks using different models such as RF, DT and SVM. This layer was trained on a real-time dataset and RF provided the best accuracy and was therefore used for the first layer in the end. On the other hand, the second layer was application-specific which was trained and tested on a synthetic dataset, which introduced new parameters specific to the application. It also used the same models when training and testing as the generalised layer. The best performer in this layer was a Bagging method-based model. This layer was used if the abnormal request bypassed the first layer. The authors evaluated this approach, resulting in the generalised layer achiev-

ing an accuracy of 91.63%. In contrast, the application-specific layer achieved an accuracy of 97.77%, which results in an overall accuracy of 96.54% with an F1-score of 0.97. They concluded that this approach outperformed related works and was considered an effective mechanism for detecting anomalies in API traffic.

Another approach made specifically towards API traffic in a microservice architecture was an SVM approach made by Baye et al. [8], rather than the RF and Bagging approach Sowmya et al. [30] took. Baye et al. also acknowledge the challenges of APIs generating large amounts of traffic and fear that manually analysing these logs would not be feasible. By employing the SVM approach on a synthetic dataset, the authors claimed that abnormal API traffic could be detected with the support of several surveys. A Gaussian distribution outlier detection technique created the labelled synthetic dataset simulating real-world API logs where normal data was defined as the traffic within the standard deviation range (σ). They also aimed to find the optimal error term to balance accuracy and false positives. They implemented SVM using the Python library scikit-learn.

The dataset had attributes such as *client IP address*, *HTTP request type*, *HTTP status code*, *bandwidth consumption*, *number of consecutive requests* and *connection duration*. The results showed that unseen anomalies were detected 83.5% of the time with an F1-score of 0.964. Meanwhile, rule-based techniques such as Snort, an Intrusion Detection System (IDS), reached a detection rate of just 20.45% with an F1-score of 0.751. The achieved false-positive rate for the SVM approach was 7.3%, making it a viable option for real-world use. With the support of several related works, SVM will be one of the evaluated approaches for this thesis, but in an unsupervised manner by using OCSVM.

Ansari [54] evaluated several unsupervised approaches to anomaly detection. According to the author, there are different ways of using unsupervised anomaly detection which is *small percentage outlier detection* and *clustering techniques*. Which one is the best option has to be evaluated. This was done by studying how different unsupervised anomaly detection algorithms applied to the raw API data would perform, considering different scenarios. The specific approaches used were OCSVM, *Density-Based Spatial Clustering of Applications with Noise (DBSCAN)* and *K-means*. Different unsupervised anomaly detection techniques were used depending on the features in the raw API data. If the data was categorical, it was concluded that it was time-series data. Therefore, the DBSCAN approach was utilised in this case. However, if the raw API data was numerical, the data could either be time-series or non-time-series. If the data was time-series data, OCSVM was utilised. Meanwhile, if the data was non-time-series data, the clustering technique K-means was used.

Overall, the stages Ansari followed for the decision-making of the unsupervised approaches started by inputting the raw data which resulted in the data being clustered into N categories where the size of the data was large. This led to the next step, which was feature selection and transformation. This step involved selecting specific features from the dataset and transforming the dataset into the required format. The features were extracted to then be used to train the chosen models. This brought Ansari to the next step, which was to train the models. Depending on the features being trained, the appropriate unsupervised model was selected.

Finally, the results could be obtained which included different clusters and categories from the data. Each cluster contained data with similar features. It was concluded that clusters with the least density were either noise or anomalies. The same conclusion was drawn for data that did not fit the normal pattern from the dataset. The results obtained showed that the chosen approaches could detect anomalies. OCSVM and DBSCAN could identify anomalies with time-series data while K-means performed efficiently with the numeric features. Ansari proposed future work to analyse these approaches further and make a more user-friendly approach to detect anomalies in an API dataset.

3.4 Anomaly Detection Frameworks

Several anomaly detection frameworks have been proposed that can be used for log analysis. These frameworks have been evaluated and compared to other frameworks and approaches. Xu et al. [55] introduced an unsupervised approach to detect log anomalies in a large-scale setting. The approach uses a framework named *AlFinder*, which supports several anomaly detection algorithms. The approach this framework took was composed of two steps. First, the framework would extract all log event templates from the given log set, which usually included clustering. This would group similar logs into a single cluster which would then be extracted together containing the event template (constant part) and its specific parameters (variable part). Continuing, the second step involved the framework's *Anomaly Detector*. This was seen as a classification problem at this stage, where each log event was defined as a class. These two steps combined would lead to the implementation and experimentation of the framework.

They used the Los Alamos National Laboratory (LANL) cybersecurity dataset, which contained over one billion entries of logs. They evaluated the effectiveness of three of the available anomaly detection algorithms. The results showed that their framework and the experiments on the chosen anomaly detection algorithms were effective, achieving an F1-score of a minimum of 70% for each algorithm. The evaluation was based on the precision and recall of the different algorithms.

Another evaluated framework proposed by Lupton et al. [56] introduces an open-source, component-based pipeline framework that could be used for anomaly detection in logs. The approach implements commonly used parsing methods as well as anomaly detection methods as replaceable components. They evaluated the framework's performance compared to rule-based methods to assess whether modern anomaly detection methods were suitable for industry system monitoring. Lupton et al. concluded that using anomaly detection in a real-world setting has the potential to detect anomalies better than rule-based methods, but it has a few challenges. These challenges include high false positive rates, which were also mentioned by Franzén et al. [51], and long training times. This opens up further work to enable the industrialisation of anomaly detection methods in logs, which this thesis will evaluate. But also, how these anomalies can be connected to API endpoints and their vulnerabilities.

3.5 Summary of Related Work

In this section, a summary of the related work will be presented in Table 3.3, 3.4 and 3.5. The different evaluation matrices are commonly used metrics in an abbreviation form, such as Precision (P), Recall (R), F-score (F1) and Accuracy (A).

Table 3.3: Summary of Current Tools and Techniques

Author(s)	Technique(s)	Description	Metrics	Dataset(s)
He et al. [33]	LR, DT, SVM, LG, PCA, IM	The authors evaluated three supervised approaches and three unsupervised approaches on two publicly available datasets.	P, R, F1	HDFS, BGL
Franzén et al. [51]	K-means	The authors performed unsupervised anomaly detection on a publicly available dataset by creating a PoC.	P, R, F1, A	HDFS
Kumari et al. [32]	K-means	The authors used K-means together with big data tools such as Apache Spark to perform unsupervised anomaly detection on the given dataset.	-	KDD Cup 1999

Table 3.4: Summary of the techniques tailored to API traffic anomaly detection

Author(s)	Technique(s)	Description	Metrics	Dataset(s)
Sowmya et al. [30]	RF, DT, SVM, KNN, etc.	The authors used a two-layer anomaly detection framework they created, where the first layers consisted of a mix of ensemble methods and classification approaches.	F1, AUC-ROC, TP, TN, FN, FP	[57], Synthetic
Baye et al. [8]	SVM	The authors generated API logs based on real traffic and used Gaussian distribution to create the training dataset for their supervised approach.	F1, FP, FN	Synthetic
Ansari [54]	DBSCAN, OCSVM, K-means	The author used three different unsupervised anomaly detection approaches on a dataset. Different models were used depending on the data type, to play into the model's strengths.	-	-

Table 3.5: Summary of anomaly detection frameworks

Author(s)	Technique(s)	Description	Metrics	Dataset(s)
Xu et al. [55]	AIFinder	The authors created an unsupervised anomaly detection framework which processes security logs in a real-time manner with acceptable precision and recall.	P, R	LANL
Lupton et al. [56]	Encoders, Detectors	The authors introduce an open-source, component-based pipeline framework for anomaly detection. They compare it to rule-based methods.	FP	Real-world incident log data



4 Method

This chapter will describe the method planned to be used for the thesis. It will describe the dataset to be used and the overall evaluation process to perform unsupervised anomaly detection in API logs.

4.1 Dataset

HTTP Dataset CSIC 2010 [58, 59] is the dataset that was used for this thesis, found by the works of Chakir et al. [60]. The dataset contains automatically generated web requests towards an e-commerce website and was developed at the *Information Security Institute* of CSIC (Spanish Research National Council). A summary of key characteristics of the dataset is presented in Table 4.1. In the web application used to create the dataset, users can e.g. buy items, add them to their shopping cart and provide personal information. The dataset contains 61000 entries, where 36000 are normal requests while 25000 are anomalous requests. Each request is therefore labeled as either normal or abnormal.

The data was generated by Giménez et al. [58] using a process that first involved collecting real data for all web application parameters. The authentic data, including names, surnames, addresses, etc., were extracted from real databases. These values were then stored in two separate databases: one containing normal values and another containing anomalous values, designed to simulate attack patterns. Furthermore, a list of all publicly available web application pages was compiled. Subsequently, for each web page, normal and abnormal HTTP requests were generated. The normal requests had parameters filled with data randomly selected from the normal database and abnormal requests were constructed with values selected from the anomalous database to simulate various attack scenarios.

For the requests classed as anomalies, three categories were crafted. Firstly, static attacks involve requests for hidden or non-existent resources, such as obsolete files, session IDs in URL rewrites, configuration files, default files etc.. Secondly, dynamic attacks entail the alteration of valid request arguments. These include but are not limited to, SQL injection, Carriage Return Line Feed (CRLF) injection, cross-site scripting, and buffer overflows. Lastly, unintentional illegal requests were also simulated. Although these requests are not inherently malicious, they deviate from the normal operation of the web application and possess a dissimilar structure compared to normal parameter values, such as a telephone number composed of letters.

Table 4.1: Summary of the dataset

Aspect	Value
Dataset size	61000
Normal entries	36000
Anomalies	25000
Class distribution	Normal: 59%, Anomalies: 41%
Origin	Giménez et al. [58]
Data types	Mixed (categorical, numerical, textual)

The columns used from the Comma Separated Values (CSV) version of the dataset compiled by Spangler [59] are presented in Table 4.2, have been chosen with specific considerations towards unsupervised anomaly detection in API logs, apart from the classification column, particularly focusing on API endpoints. The motivation behind this selective inclusion originates from each column’s characteristics on understanding web application behaviour and security nuances which resemble API logs characteristics. However, valuable data such as timestamps, status codes and response time are absent from the dataset. These implications and limitations are discussed in Chapter 6.

Analysing the *method* and the *url* column forms the foundation of understanding client-server interactions within API ecosystems. The URL, partly defining the API endpoint, indicates the specific resource or operation targeted within the web application. Accompanying HTTP request methods such as GET and POST detail the action requested on these endpoints, making the combination of *method* and *endpoint* crucial for anomaly detection. For example, unexpected POST requests could signal unauthorised data modifications, whereas a surge in POST requests might denote a data breach or injection attempts. This holistic view of method and endpoint interactions offers great insights into potential security vulnerabilities, emphasising the importance of using both for signs of exploitation.

Furthermore, incorporating *host* information once encoded for analysis helps identify requests to potentially uncertain hosts or those frequently targeted by cyber-attacks, adding a layer of contextual understanding. Similarly, analysing the *accept* header through encoding enlightens the Multipurpose Internet Mail Extension (MIME) types a request is prepared to handle, with anomalies potentially indicating exploitation efforts.

The *content_length* metric provides immediate insight into the request or response body’s size, facilitating the detection of anomalous payloads that could suggest attempts at exploitation, such as buffer overflow attacks. Moreover, analysing the request *content* through various derived features like character counts and patterns exposes direct and intricate indicators of malicious intent, ranging from SQL injection to Cross-Site Scripting (XSS) attacks.

Table 4.2: Description of the used CSIC log columns

Column	Description
<i>method</i>	The HTTP method used (e.g. GET, POST).
<i>host</i>	The domain of the request.
<i>accept</i>	The content types that the client can understand.
<i>content_length</i>	The length of the request.
<i>content</i>	The actual content of the request.
<i>url</i>	The URL (API endpoint) requested to.
<i>classification</i>	Labels representing the same thing as ‘class’, used for evaluation metrics.

4.2 Anomaly Detection Process

For this thesis, the methodology will follow a four-phase strategy inspired by Ahmed et al. [19], depicted in Figure 4.1. The four phases are *Preparation*, *Anomaly Detection*, *Output* and *Evaluation*. The solid lines represent the steps taken when performing the evaluation, while the dashed lines exclude steps from the procedure presented by Ahmed et al.

First, eligible data as input for training and testing is mandatory. This input is usually either a dataset that can distinguish between benign and malicious activities or raw logs [19]. The input data often needs to be pre-processed with techniques tailored to accommodate the unique characteristics of the dataset. For example, IP addresses exhibit a hierarchical structure, port numbers are represented numerically, and protocols are categorised into distinct types using e.g. encoders. The processing methods rely on the selected anomaly detection techniques, broadly classified as supervised or unsupervised. For this thesis, only unsupervised anomaly detection was explored as mentioned earlier. These anomaly detection techniques are then applied to the data, generating an output which can be graded using scores or labels [19]. This thesis will only consider scores since labels typically apply to classification problems which does not apply here. These scores are then used to evaluate the entire process. Should the result based on the evaluation be deemed poor or that there are improvements to be made, an iterative approach can be taken to allow tuning the models for optimal performance on the dataset.

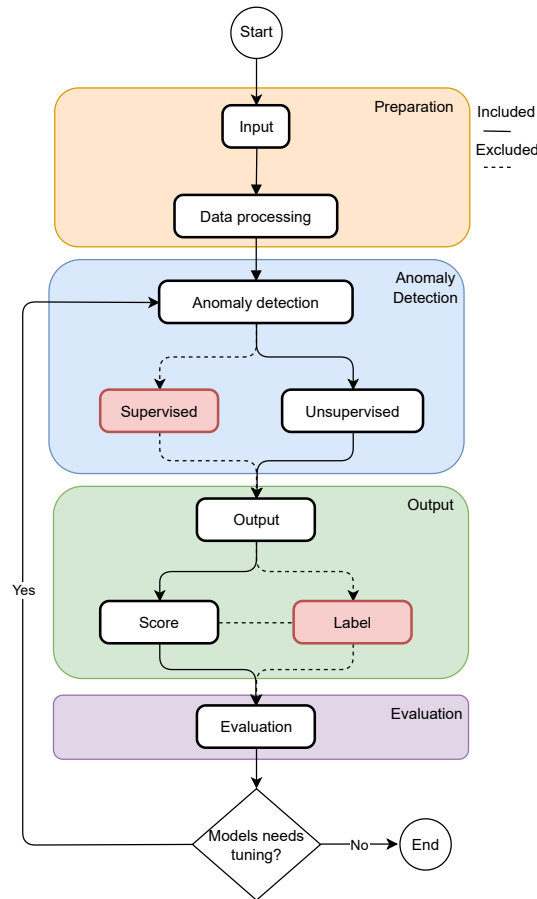


Figure 4.1: Proposed process of anomaly detection

4.2.1 Preparation Phase

The preparation phase can vary significantly depending on the data pre-processing required. The preparation phase involves tasks such as *data collection*, *feature extraction* and sometimes *data normalisation* or *data standardisation* as described by various authors [37, 40, 49]. In Figure 4.2 [38], the appropriate tasks for the preparation phase are illustrated, consisting of steps *log collection*, *feature extraction*, *data normalisation* and *machine learning*. These steps were taken when performing unsupervised anomaly detection. However, normalisation and standardisation are not always required. But normalisation was performed in this thesis to improve prediction accuracy potentially and to visualise some of the models, as demonstrated by Lanigan et al. [61].

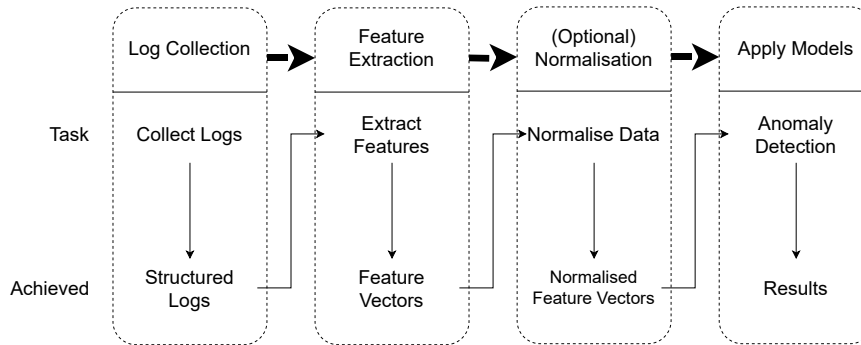


Figure 4.2: Proposed preparation phase flow

Input

The initial input gathering, a pivotal task for log collection, often wavers between real-world and synthetic data. The scarcity of real API log data for public use is attributed to several factors. Confidentiality policies of businesses, the size of datasets, the number of API endpoints, the nature of imbalanced data and inconsistencies in data formatting are primary concerns [30, 39]. These limitations amplify the significance of projects like LogHub [50], which aggregates 19 log datasets amassing over 77 GB, catering to academic and industry interests. For this thesis, however, the identified challenges render the publicly available logs from LogHub inapplicable for API log analysis. Instead, the CSIC HTTP 2010 Dataset [59], explained in Section 4.1, is considered suitable for interpretation as API logs, thus being selected as the primary dataset of this master thesis despite its limitations, as is discussed later.

Data processing

With structured logs collected from the selected dataset, initial processing of the data is performed. The main reason is that different columns in the dataset contain other data types, such as numerical and categorical data in text format. To use anomaly detection models properly, all columns must be in numerical format as described by Juvonen et al. [46]. Therefore before model training, the dataset underwent a pre-processing procedure to ensure optimal feature representation and compatibility with ML algorithms for unsupervised anomaly detection.

Initially, data cleaning operations were performed as presented in Listing 4.1, particularly on the *content_length* attribute, where non-numeric characters were extracted and discarded, transforming the feature into a numeric datatype. Any missing values encountered during this transformation were subsequently assigned with zero, assuming that a missing content length equates to an absence of content. A similar approach was applied to the *accept* attribute, extracting numeric content and handling non-numeric or missing data by assigning a default value. This procedure follows similar measures taken by Khan et al. [62].

```

1 X['content_length'] = X['content_length'].astype(str)
2 X['content_length'] = X['content_length'].str.extract(r'(\d+)')
3 X['content_length'] = pd.to_numeric(X['content_length'], errors='coerce').fillna(0)
4 X['accept'] = X['accept'].astype(str)
5 X['accept'] = X['accept'].str.extract(r'(\d+)')
6 X['accept'] = pd.to_numeric(X['accept'], errors='coerce').fillna(1)

```

Listing 4.1: Data cleaning

Following data cleaning, feature engineering was performed to construct meaningful attributes that could capture the nuances of HTTP requests in the API. A subset of the feature engineering performed is presented in Listing 4.2. This process included deriving a variety of URL-specific features, such as the count of dots, directories, and embedded domain names, as well as the length of the URL and the presence of any suspicious terms that might indicate malicious intent. These suspicious terms that might exist are weighed based on their severity capabilities towards an API. Comparable metrics were computed for the *content* attribute, including counts of various characters and evaluating the content's encoding status. Additionally, the categorical *method* attribute was numerically encoded to facilitate its integration into the subsequent modelling efforts.

```

1 X['count_dot_url'] = X['url'].apply(count_dot)
2 ...
3 X['special_count_url'] = X['url'].apply(count_special_characters)
4 X['count_dot_content'] = X['content'].apply(apply_to_content, function=count_dot)
5 ...
6 X['is_encoded_content'] = X['content'].apply(apply_to_content, function=is_encoded)

```

Listing 4.2: Feature engineering

An exploratory data analysis was conducted to gain deeper insights into the dataset's structure and characteristics. This exploration included identifying the most frequently occurring URLs and enumerating unique values across different features, thereby establishing an understanding of the data's diversity and distribution. Non-numeric fields such as *method*, *host*, and *accept* underwent label encoding, converting them into numeric codes to enable processing by ML models, presented in Listing 4.3. The *classification* attribute, a binary representation of normal and abnormal requests, was isolated as the target variable for model training.

```

1 lb_make = LabelEncoder()
2 X["method_enc"] = lb_make.fit_transform(X["method"])
3 X["host_enc"] = lb_make.fit_transform(X["host"])
4 X["accept_enc"] = lb_make.fit_transform(X["accept"])

```

Listing 4.3: Label encoding

In the end, the pre-processing phase assembled a refined feature set designated as *X_preprocessed* containing labels representing the pre-processing and feature engineering performed as presented in Listing 4.4. This set comprised the newly engineered features and the label-encoded categorical variables, ensuring a rich dataset ready for the training of the anomaly detection model. The pre-processing pipeline was carefully designed to maintain the integrity of the dataset, thus establishing a robust foundation for the upcoming phases of the anomaly detection process.


```

1 labels=['count_dot_url', 'count_dir_url', 'count_embed_domain_url', 'count-http', '
  count%_url', 'count?_url', 'count-_url', 'count=_url', 'url_length', '
  hostname_length_url', 'sus_url', 'count-digits_url', 'count-letters_url', '
  number_of_parameters_url', 'is_encoded_url', 'special_count_url', '
  unusual_character_ratio_url', 'method_enc', 'host_enc', 'accept_enc', '
  count_dot_content', 'count%_content', 'count-_content', 'count=_content', '
  sus_content', 'count_digits_content', 'count_letters_content', 'content_length', '
  is_encoded_content', 'special_count_content']
2
3 X_preprocessed = X[labels]

```

Listing 4.4: Labels used after data processing task

For all models except IF, since it does not need to utilise PCA, the data is also scaled as given in Listing 4.5 and Listing 4.6. This is done to emphasise the difference in features to detect anomalies with better accuracy as described by Astekin et al. [38] and investigated by Lanigan et al. [61]. It is also used to decrease the time complexity since computing many dimensions is a time-consuming action [33, 51]. StandardScaler does the scaling from Scikit-learn and performs Z-score normalisation as defined in Section 2.8.

```

1 scaler = StandardScaler()
2 X_preprocessed_scaled = scaler.fit_transform(X_preprocessed)
3
4 pca = PCA(n_components=2, random_state=42)
5 pca.fit(X_preprocessed_scaled)
6
7 X_train_pca = pca.transform(X_preprocessed_scaled)

```

Listing 4.5: Scaling data using PCA for clustering

```

1 scaler = StandardScaler()
2 X_train, X_test, y_train, y_test = train_test_split(X_preprocessed, y, test_size
  =0.3, random_state=42)
3
4 X_train_scaled = scaler.fit_transform(X_train)
5 X_test_scaled = scaler.fit_transform(X_test)
6
7 pca = PCA(n_components=2, random_state=42)
8 pca.fit(X_train_scaled)
9
10 X_train_pca = pca.transform(X_train_scaled)
11 X_test_pca = pca.transform(X_test_scaled)

```

Listing 4.6: Scaling data using PCA for OCSVM

4.2.2 Anomaly Detection Phase

The anomaly detection phase starts when the preparation phase is done and data is ready to be fed to the anomaly detection models. This is the most straightforward phase since the only manual task performed is feeding the feature matrix to the selected anomaly detection models for training and testing [30, 49]. The tasks to be performed are model selection and applying the selected models to the pre-processed data to generate an output.

Model Selection

With the pre-processed data ready, models for the unsupervised anomaly detection process were selected. The selection of models depends on several factors, such as the characteristics of the datasets used, the findings from relevant literature described in Section 3.1 and what could be interesting to evaluate based on other findings outside of the described literature but still relevant. The models selected were K-means, GMM, IF and OCSVM.

K-means and GMM are clustering approaches evaluated previously with various results [34, 35, 51] as well as found in a survey by Eltanbouly et al. [45]. By using previously used models, it can be concluded how well they work on API logs. K-means and GMM were also used by Sánchez-Zas et al. [21] to evaluate unsupervised anomaly detection, as they are among the best-known learning models to allow the grouping of data into clusters. To group K-means and GMM in an interpretable way, optionally visualising them and to decrease the computation time of OCSVM due to the high dimension count on the dataset, PCA was used as a dimensionality reduction model, as described in Section 2.4.2.

From general findings, IF and OCSVM will also be used for evaluation. Both models are unsupervised outlier detection models where IF is an ensemble-based model while OCSVM is part of the SVM family of algorithms. IF was selected as a model to evaluate unsupervised anomaly detection because it acted as an unsupervised outlier detection approach [63] and has been previously evaluated with promising results [34]. OCSVM was selected due to its promising results performed in previous studies by Ansari [54] and Eltanbouly et al. [45].

Initial Parameter Settings for Models

When the models are first used, minimal parameter tweaking is performed to see the baseline performance of the selected models. Models and their parameters are presented in Listing 4.7. To ensure the reliability of the baseline models, a specific random state is set to provide a deterministic behaviour. An exception for this is made for the OCSVM model since it does not require a set random state to be replicated.

```

1 # K-means baseline model
2 kmeans = KMeans(n_clusters=2, random_state=42)
3
4 # GMM baseline model
5 gmm = GaussianMixture(n_components=2, random_state=42)
6
7
8 # Isolation Forest baseline model
9 iso_forest = IsolationForest(n_estimators=100, contamination=0.1, random_state=42)
10
11
12 # OCSVM baseline model
13 ocsvm = OneClassSVM()
```

Listing 4.7: Baseline models

Tuning the Parameters Based on Analysis

After the initial run of the models, various analysis tasks was performed to find optimal values for the models depending on the dataset such as elbow point analysis, Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC) analysis as well as True Positive Rate (TPR) and manual inspection of the visualisation of the data where possible using e.g. confusion matrices and Receiver Operating Characteristic (ROC)-Area Under Curve (AUC)-curves. With subsequent runs performed, the optimal parameters found for the models are presented in Listing 4.8.

```

1 # K-means tuned model
2 kmeans = KMeans(n_clusters=5, random_state=42)
3
4 # GMM tuned model
5 gmm = GaussianMixture(n_components=18, random_state=42)
6
7
8 # Isolation Forest tuned model
9 iso_forest = IsolationForest(n_estimators=300, contamination=0.5, random_state=42)
10
11
12 # OCSVM tuned model
13 ocsvm = OneClassSVM(kernel='rbf', nu=0.5, gamma='auto')

```

Listing 4.8: Tuned models

Application of Anomaly Detection

When the models for unsupervised anomaly detection are selected and their parameters set, it is time to apply and generate their output from the pre-processed dataset. When the models are applied to the dataset in this thesis, all models except IF will utilise a scaled version of the dataset. The reason for this is that the mean and standard deviation scale of the dataset does not affect the performance of IF while it does affect the performance of K-means, GMM and OCSVM as well as PCA.

4.2.3 Output Phase

In this phase, the results from the anomaly detection phase are generated and obtained. Depending on the models used, the results can be presented in different ways, both in the output and visually. The output can normally be split into two parties, scoring-based and label-based (i.e. binary-based), as presented in Figure 4.1. This thesis will utilise the scoring-based models but translate the scoring to binary decisions for evaluation in the next and final phase. The selected models K-means, GMM, IF and OCSVM are scoring-based techniques but the output provided are the same.

4.2.4 Evaluation Phase

The final phase using anomaly detection in API logs is the evaluation. The main task for this phase is to evaluate the results achieved by the output in the previous phase. This can be done in multiple ways, similar to the output phase depending on the chosen anomaly detection approach. Using applicable metrics can lead to insightful analysis and discussions of what benefits and drawbacks exist in the current model. Depending on the model and data at hand, different metrics can be utilised to determine its performance in detecting anomalies on the used dataset. These metrics were labelled as the performance metrics for this thesis to answer the second research question. Also, the effect of tuning the model to achieve an optimal model is analysed in this phase.

Time was measured for all models to finalise the answer to the second research question. This provide insight as to how fast the models are trained and compute anomalies in the dataset, which can be a crucial metric for deciding whether a model is worth implementing in the future. To be consistent with calculating the time, the time was calculated for how long it takes to fit the model considering the given parameters to the model.

Evaluating the Clustering

For the clustering models used for unsupervised anomaly detection, K-means and GMM, the first valid approach to evaluate the clustering quality achieved. There are multiple metrics to measure this.

The Silhouette coefficient is the first measure used to assess the quality of a clustering model. It provides insight into the cohesion within clusters and the separation between clusters, thereby indicating how well each data point is classified within its cluster. To compute the Silhouette value for a given data point, one employs the mathematical expression as depicted in Equation 4.1, where i represents a data point, a denotes the mean intra-cluster distance, and b represents the mean nearest-cluster distance. To calculate the Silhouette for a data point i , two calculations are required:

- a : This represents the average distance between a data point i and all other points within the same cluster as i . In simpler terms, it measures how close i is to the other points in its cluster. Also known as the mean intra-cluster.
- b : This represents the average distance between a data point i and all points in the nearest neighbouring cluster to i . In simpler terms, it measures how far i is from the points in the closest neighbouring cluster. It is also known as the mean nearest-cluster distance.

The Silhouette coefficient ranges between -1 and 1 , where higher values indicate well-assigned data (high cohesion within clusters and significant separation between clusters), and lower values suggest otherwise [21].

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad (4.1)$$

One can use extrinsic metrics such as Adjusted Rand Index (ARI) and Normalised Mutual Information (NMI) to indicate how well the cluster groupings are. ARI, used by Ariyaluran Habeeb et al. [41], is an enhancement of the Rand index. It adjusts the Rand Index by considering the expected similarity of all pairings between clusterings due to chance, which provides a more robust measure. The ARI is calculated by using Equation 4.2, where Index is the Rand Index, Expected Index is the Rand Index expected due to random chance, and Max Index is the maximum possible Rand Index. Additionally, NMI measures the mutual information between two clusterings as a fraction of the geometric mean of their entropies, thus normalising the mutual information. The formula for NMI is presented in Equation 4.3, where $I(X; Y)$ is the mutual information between the cluster assignments X and Y , and $H(X)$ and $H(Y)$ are the entropies of X and Y respectively. This measure is useful for comparing clusterings that may have a different number of clusters.

$$\text{Adjusted Rand Index (ARI)} = \frac{\text{Index} - \text{Expected Index}}{\text{Max Index} - \text{Expected Index}} \quad (4.2)$$

$$\text{Normalised Mutual Information (NMI)} = \frac{2 \times I(X; Y)}{H(X) + H(Y)} \quad (4.3)$$

When using clustering as an approach, there is a risk that high dimensional data is used. When this is visualised, the results will not be represented properly or are challenging to interpret. To better illustrate the data, one can use PCA as described in Section 2.4.2 as a dimensionality reduction technique to reduce the number of dimensions but preserve as much information as possible from the original feature space [21].

For K-means, a specific valuable metric is computing the *Within Set Sum of Squared Error* (WSSSE), also known as *inertia*. When visualising clusters, a scatter plot is usually how

clusters are represented. However, should three or more features exist to be plotted, the number of dimensions would have to be reduced using a dimensionality reduction technique. This could imply a loss of information. Using WSSSE as a metric, it is possible to find out the total error of the distance of the data relative to the centroid to which the data belongs.

The WSSSE is computed by summing up the squared Euclidean distances of each data point within its respective cluster centroid, presented in Equation 4.4. Here, K represents the total number of clusters, S_k is the set of all data points in cluster k , x denotes an individual data point within cluster S_k and μ_k is the centroid of cluster k . The term $\|x - \mu_k\|^2$ calculates the squared Euclidean distance between the data point x and the centroid μ_k , summer overall dimensions. Lower WSSSE values indicate the clusters are tighter and more compact, which is typically desired for effective clustering.

$$\text{WSSSE} = \sum_{k=1}^K \sum_{x \in S_k} \|x - \mu_k\|^2 \quad (4.4)$$

When studying WSSSE, one can study the *elbow point* of the graph. The motivation is to find the optimal value of the number of clusters, to maximise the results with minimal loss of information and complexity. By studying Figure 4.3, different error values depending on the number of clusters used are presented. The figure suggests that the optimal value for the model is $k = 4$, which is the elbow point [21].

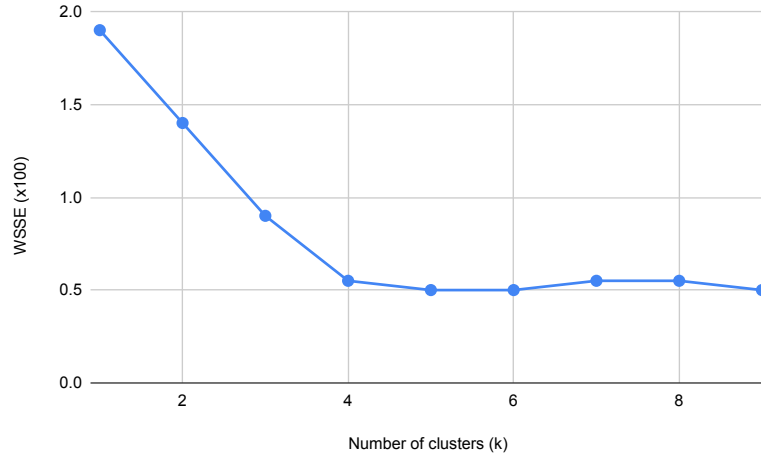


Figure 4.3: WSSSE (Inertia) error graph with elbow point at $k = 4$

As for GMM, one can calculate the specific metrics AIC and BIC to evaluate the optimal number of components regarding the dataset's characteristics to avoid overfitting the models. AIC is a measure used for model selection that balances the model fit and its complexity, presented in Equation 4.5. Here, k is the number of estimated parameters in the model and $\hat{L}$ is the maximum value of the likelihood function for the model. BIC is another measure which considers a balance between the model fit and its complexity, but penalising models with more parameters, presented in Equation 4.6. Here, n is the number of observations, k is the number of parameters estimated by the model and $\hat{L}$ is the maximum likelihood of the model [35]. Both metrics strive for lower values, indicating a better balance between the model fit and the complexity.

$$\text{AIC} = 2k - 2\ln(\hat{L}) \quad (4.5)$$

$$\text{BIC} = \ln(n)k - 2\ln(\hat{L}) \quad (4.6)$$

Guo et al. [35] proposed using BIC since it adds a penalty depending on the number of parameters to the log-likelihood to overcome overfitting issues. For this thesis, both AIC and BIC were used to get a more comprehensive evaluation of the balance of the different versions of the model.

When using GMM models, another useful metric used by e.g. Zhou et al. [64] is utilising the Log-Likelihood (LL), which tells us how well the model explains the observed data. Higher LL values suggest that the model parameters (θ) are more likely to have generated the observed data, indicating a better alignment between the model and the actual data distribution. In the context of GMM tuning, maximising the LL is often a primary objective as it signifies an enhancement in the model's capability to accurately represent the underlying data distribution. The LL is calculated using Equation 4.7, where N represents the total number of data points, K denotes the number of mixture components in the GMM, α_k signifies the weight of the k -th Gaussian component, and $\phi(x|\theta_k)$ denotes the probability density function of the k -th Gaussian component evaluated at data point x .

$$\log L(\theta) = \sum_{j=1}^N \log \left(\sum_{k=1}^K \alpha_k \phi(x|\theta_k) \right) \quad (4.7)$$

Evaluating the Models

To identify the anomalies using clustering, Farschi [36] proposed that data instances can be labelled as anomalies if the data does not belong to any cluster, is far away from their cluster centres or belongs to sparse clusters. For this thesis, the approaches for anomaly detection are tailored to the characteristics of each clustering method used.

For K-means, anomalies are marked as data points far from the cluster centres. This choice is inspired by the method employed by Franzén et al. [51], where a percentile threshold, the top 50% of the data points for this thesis, is used to define anomalies. This threshold is selected based on the anomaly ratio present in the dataset and was further discussed in Chapter 6. In GMM, anomalies are identified using the log-likelihood scores from the model. Each point is evaluated based on its probability density under the model. Data points with significantly low probability densities, determined by a 50th percentile threshold, are considered anomalies. This leverages the assumption that normal data points fit well within the Gaussian distributions defined by the GMM. Anomalies are likely found in low-probability regions of the fitted Gaussian components or in association with sparsely populated clusters.

The values for precision, recall, F1-score, AUC and accuracy in the tables presented in this chapter represent the model's capabilities towards the anomaly class, i.e. how well they detect anomalies. Because of the available labels for each request telling whether a request is normal or an anomaly, all models can utilise commonly used metrics such as precision, recall and F1-score as well as ROC-AUC score and accuracy to validate the effectiveness of the models. These metrics are crucial in assessing how well the models detect anomalies, which is the focus of our evaluations. Precision is presented in Equation 4.8, recall in Equation 4.9 and F1-score in Equation 4.10 [33, 40, 44].

K-means and GMM can use the labelled data and compare them to the determined anomalies using the previously mentioned percentile threshold. For IF and OCSVM, a prediction and decision function from each model is used to compare to the ground truth labels. Some specific measurements recommended by Muriti et al. [65] and Khan et al. [62] are accuracy and detection rate. Accuracy is a widely used metric to determine the effectiveness of certain anomaly detection models and the formula is presented in Equation 4.11.

$$\text{Precision (P)} = \frac{TP}{TP + FP} \quad (4.8)$$

$$\text{Recall (R)} = \frac{TP}{TP + FN} \quad (4.9)$$

$$\text{F1-score (F1)} = 2 \times \frac{P \times R}{P + R} \quad (4.10)$$

$$\text{Accuracy (A)} = \frac{TP + TN}{TP + FP + TN + FN} \quad (4.11)$$

According to the previous studies on anomaly detection and the detection rate, the outcomes can be categorised into four types: True Positive (TP), True Negative (TN), False Positive (FP) and False Negative (FN). These categories are fundamental in evaluating the performance of detection models.

The performance metrics can be visualised using ROC curves, as illustrated in Figure 4.4. The ROC curve is a graphical representation of a model's diagnostic ability, and the AUC quantifies the overall ability of the model to discriminate between the classes across all thresholds. The AUC represents the degree of separability, with values closer to 1 indicating higher model accuracy. A model performing at a level equivalent to random guessing is represented by an ROC value of 0.5, depicted as a diagonal line in the ROC space.

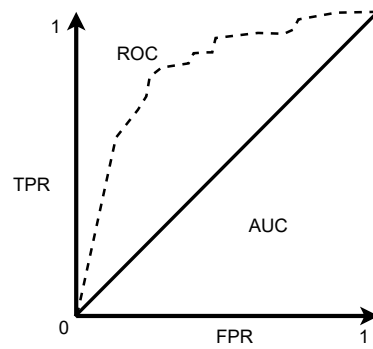


Figure 4.4: Example of a ROC-AUC curve

Confusion matrices can also be used, providing a straightforward visualisation of model performance, including TPR and other metrics, as shown in Figure 4.5. These matrices can depict the ratio of how well the models predict true anomalies (TP), true normal (TN), false anomalies (FP) and false normal (FN). For anomaly detection, it is heavily sought after achieving a low FP and FN ratio.

True label	Normal	True Negative (TN)	False Positive (FP)
	Anomaly	False Negative (FN)	True Positive (TP)
		Normal	Anomaly
		Predicted label	

Figure 4.5: Example of a confusion matrix

5 Result

This chapter consists of the collected results from the methodology presented in Chapter 4. The different model performances from the baseline parameters will be shown. Furthermore, how the tuning and parameters were performed with different values will be showcased. Finally, the result of the performances of the tuned models will be shown. All times recorded in the results are based on the PC specs specified in Table 5.1. This indicates the power capabilities and what to expect if the results are attempted to be reproduced.

Table 5.1: PC specs used for evaluation

CPU	RAM	GPU
AMD Ryzen 7600X	G.Skill 32GB (2x16GB) DDR5 6000MHz	Radeon RX 6650 XT 8GB

5.1 Baseline Models

GMM and IF performed the best out of all the baseline models. However, IF had a high percentage of FN, indicating an urgent need for improvements. K-means and OCSVM performed worse than random guessing at certain thresholds.

The baseline models are configured with their default parameters as presented in Listing 4.7. The performance metrics for these models are shown in Table 5.2 and visually compared through ROC curves in Figure 5.1 and confusion matrices in Figure 5.2.

By first inspecting the baseline model of GMM, it can be observed that it achieved a balanced result with a short training time. A precision of 0.63 indicates that the model can find true anomalies decently. Furthermore, the recall of 0.72 suggests that GMM detected actual anomalies in the data with decent success. The F1-score of 0.67 describes a balance between precision and recall, reflecting the model's moderate effectiveness in identifying anomalies. The GMM model also has a good AUC score of 0.76, meaning that the GMM has a good ability to distinguish between normal requests and anomalies. The accuracy of 0.71 indicates that the model could correctly identify various anomalies.

K-means presents a more mixed set of results but with a shorter model training time compared to GMM. The model's precision, a modest 0.52 as shown in Table 5.2, tells that only about half of its anomaly predictions were accurate. This is further detailed by the darker

shades in the anomaly columns of the confusion matrix in Figure 5.2. Its recall of 0.62 indicates it detected a good amount of the actual anomalies. However, 38% was incorrectly predicted, emphasising the need for further refinement. The F1-score of 0.56 further suggest refinement of the model to improve the balance between precision and recall. The AUC score of 0.58 is closer to random guessing, implying a less discerning model. While the model achieved an accuracy of 0.61, the baseline K-means model may not be satisfactory for reliable anomaly detection in critical applications. This further suggests a need for model refinement or exploration of alternative algorithms that could enhance accuracy.

The IF model processes the selected dataset quickly, completing its run in only 0.18 seconds, suggesting good efficiency and indicating suitability for applications requiring rapid data processing. While its baseline model shows a high AUC, indicating a good discriminative ability, its F1-score of 0.40 suggests a potential imbalance in precision and recall which can be seen by acknowledging the precision of 0.78 and the recall of 0.27. The precision shares that the model has high TP and low FP, which can be confirmed by inspecting the confusion matrix for IF in Figure 5.2. Meanwhile, the low recall suggests that the model struggles with capturing all actual anomalies. This can be confirmed by inspecting the confusion matrix for IF in Figure 5.2, where it is visualised that the model has a high rate of FN. These observations tell that the accuracy of 0.67 does not represent the entire story, because of the imbalance in the dataset.

The OCSVM model showed promising results but suffered from a longer training time compared to the other models. A precision of 0.55 presented in Table 5.2 indicates that over half of the anomalies it detected were correct, an observation that can be further dissected in Figure 5.2. The recall of 0.66 demonstrates that the model identified most of the actual anomalies. The balanced F1-score of 0.60 suggests a fair trade-off between precision and recall. An AUC of 0.63 denotes moderate discrimination ability, and accuracy of 0.63 signifies that 63% of all requests were correctly classified by the model. There is however room for improvement in model calibration to reduce false negatives and bolster true positives.

For K-means and OCSVM, it is important to acknowledge the ROC curve at higher thresholds, specifically when the False Positive Rate (FPR) surpasses between 0.6 and 0.7. At this point, these two models perform worse than random guessing, indicating an urgent need to improve the model's anomaly detection capabilities. Performing worse than random guessing is not an applicable level for introducing anomaly detection.

Finally, interesting findings were found and can be inspected in Table A.1. For GMM, the silhouette score is reported at 0.41, suggesting moderate clustering quality. However, the ARI and NMI scores are relatively low, indicating limited agreement between the clustering results and true data labels. Analysing the anomaly information reveals high anomaly ratios within each defined cluster, suggesting challenges in distinguishing between normal and abnormal data points. K-means exhibits a higher silhouette score of 0.64, indicating better clustering quality than GMM. However, similar to GMM, the ARI and NMI scores for K-means are relatively low, suggesting sub-optimal alignment with the true clustering structure. Despite the improved silhouette score, K-means also struggles with accurately identifying anomalies, as evidenced by the high anomaly ratios within clusters.

Overall, the baseline table highlights the initial performance of K-means and GMM in clustering and anomaly detection, indicating the need for further optimisation and refinement to enhance clustering accuracy and anomaly detection capabilities.

Table 5.2: Performance metrics for baseline models

Model	P	R	F1	AUC	A	Time (s)
GMM	0.63	0.72	0.67	0.76	0.71	0.35
K-means	0.52	0.62	0.56	0.58	0.61	0.12
Isolation Forest	0.78	0.27	0.40	0.84	0.67	0.18
OCSVM	0.55	0.66	0.60	0.63	0.63	32.35

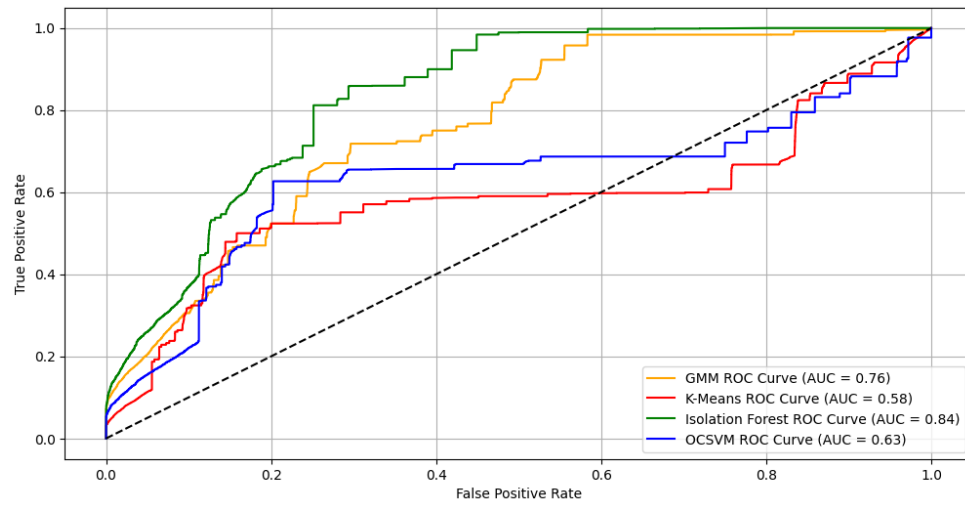


Figure 5.1: Comparison of ROC curves for baseline models

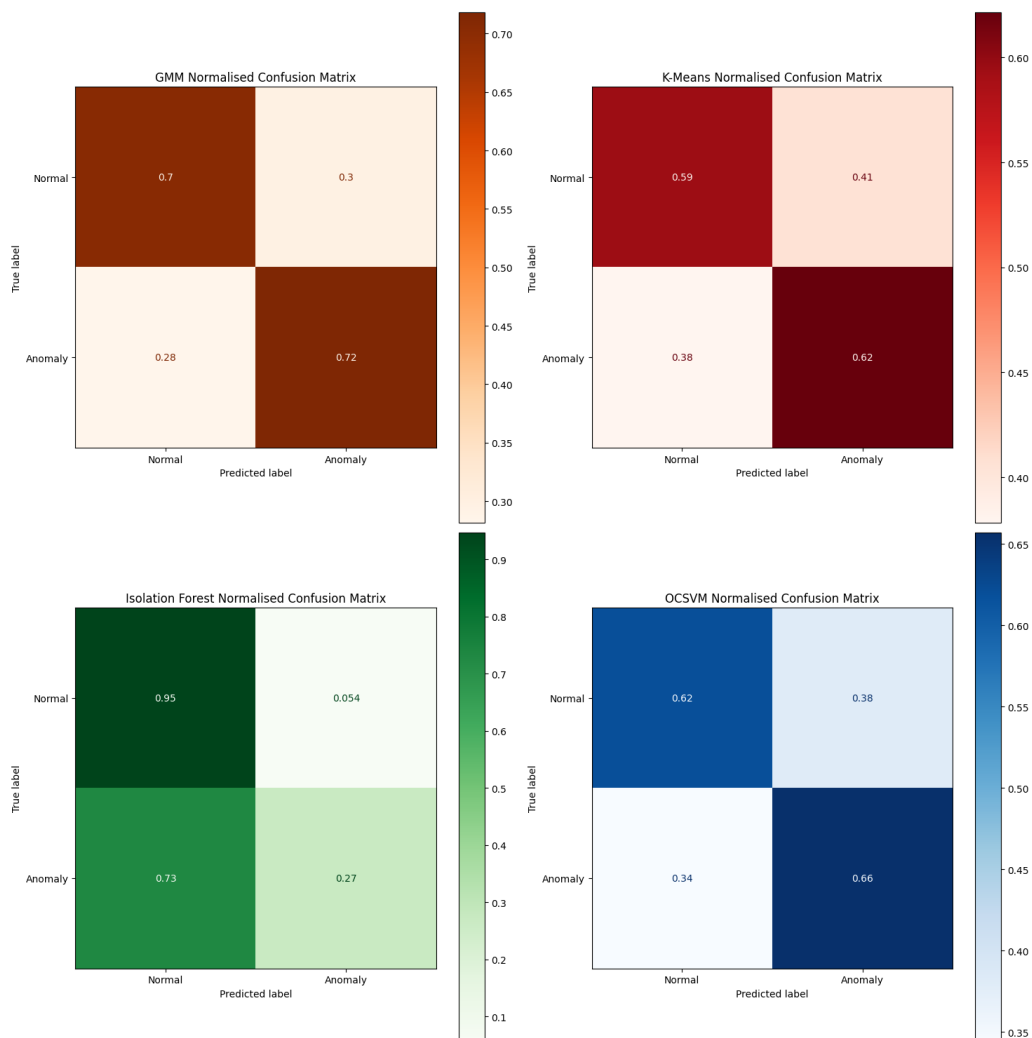


Figure 5.2: Comparison of confusion matrices for baseline models

5.2 Tuning the Models

This section presents the tuning of the models used for this master thesis, where each model has to be tuned differently. First, the process for tuning GMM is performed. Then the tuning process of the K-means model is performed. The IF model is tuned after that and finally, the tuning process of the OCSVM model is performed. Tables and figures used for the baseline models, and later on tuned models, are not visualised at this stage as there would be an overflow of information. These figures and tables would be irrelevant as the goal is to showcase the tuned model results later in this chapter.

5.2.1 GMM

Tuning a GMM involves selecting the optimal number of mixture components (Gaussian distributions) that best model the data for anomaly detection. The tuning consisted of calculating various values using different numbers of mixture components, presented in Table 5.4. Furthermore, the silhouette, AIC and BIC scores are visualised in Figure 5.3, Figure 5.4 and Figure 5.5. The scores can be analysed to assess the model complexity and fit, while the figures highlight how changes in the number of Gaussian mixture components influence model performance metrics.

Before testing the different number of mixture components, evaluating the different covariance available for a GMM model was performed. For the baseline number of mixture components, the results of using different covariance are displayed in Table 5.3. This shows that using the covariance type *full* provides the best performance when using two mixture components and is used when testing different numbers of mixture components.

The different metrics tested are outlined in Table 5.4. Silhouette scores, AIC and BIC values in a range of mixture component numbers were recorded, seeking to find the model that best balances model complexity with goodness of fit. These metrics are illustrated in Figure 5.3 for silhouette scores, Figure 5.4 for AIC, and Figure 5.5 for BIC, respectively. The silhouette score tells us that using at least 4 mixture components is vital for improved cluster structure, with the tops reaching close to 0.6 when using 5 or more than 12 mixture components. As for AIC and BIC scores, it can be observed that the more mixture components GMM uses, the lower scores are received. The decrease plateaus until more than 20 mixture components are used, where the AIC and BIC scores reach negative values. Analysing silhouette scores and the AIC and BIC scores across different mixture component numbers, a balance between cluster separation and compactness can be identified right before 20 mixture components are used.

Other metrics considered when tuning the model were LL, ARI and NMI. As the number of mixture components increases, the LL tends to increase initially, indicating a better fit between the model and the observed data. However, beyond a certain point, the LL may start to decrease or plateau, suggesting diminishing returns or overfitting as the model becomes too complex. The ARI and NMI also exhibit trends with increasing mixture components. Initially, both ARI and NMI may increase, indicating better clustering performance. However, similar to LL, they may eventually stabilise or decrease, reflecting diminishing improvements or potential overfitting. Balancing is essential for the model complexity with performance metrics to avoid overfitting and ensure generalisability to unseen data.

Inspecting the precision, recall, F1-score, AUC and accuracy, the difference depending on the number of mixture components is not affected heavily. Furthermore, the time taken slightly increases when increasing the number of components. However, this increase is insignificant so it does not affect the result heavily.

After finding the optimal number of mixture components, the different covariance options were tested again with their results showcased in Table 5.5. It can be observed that the *diag* covariance approach now performs slightly better than the *full* covariance approach, which

is the default approach for GMM, indicating that using more mixture components improves the covariance type *diag* more than *full*.

After analysing these values, the optimal number of mixture components selected for the tuned model was 18, since it provides optimal values all-round compared to the other mixture components investigated. One could make a case for 5 components. However, the difference in AIC and BIC scores is enough to select 18 mixture components as the optimal number. Furthermore, due to the slightly better performance using *diag* as the covariance type, it was chosen for the tuned model. One thing to note is the sudden change when going above 20 mixture components, the reason why none of these are selected is due to the sudden sharp change in a majority of metrics. This could be a case of overfitting and therefore not considered.

Table 5.3: GMM metrics for different covariance types with 2 components

Covariance Type	P	R	F1	AUC	A	Time (s)
Full	0.628	0.718	0.670	0.711	0.710	0.23
Tied	0.561	0.672	0.611	0.653	0.649	0.22
Diag	0.522	0.618	0.566	0.612	0.611	0.21
Spherical	0.562	0.668	0.611	0.653	0.650	0.20

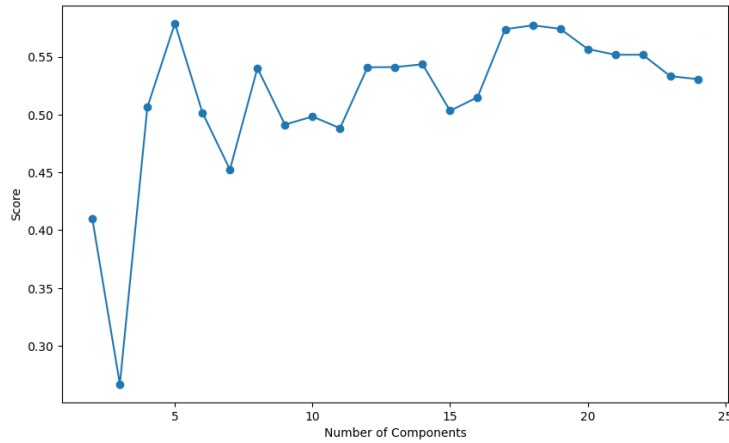


Figure 5.3: Silhouette using a different number of components

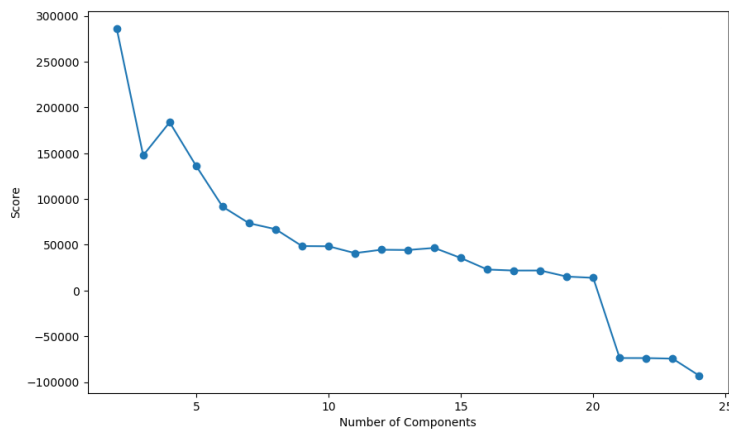


Figure 5.4: AIC using a different number of components

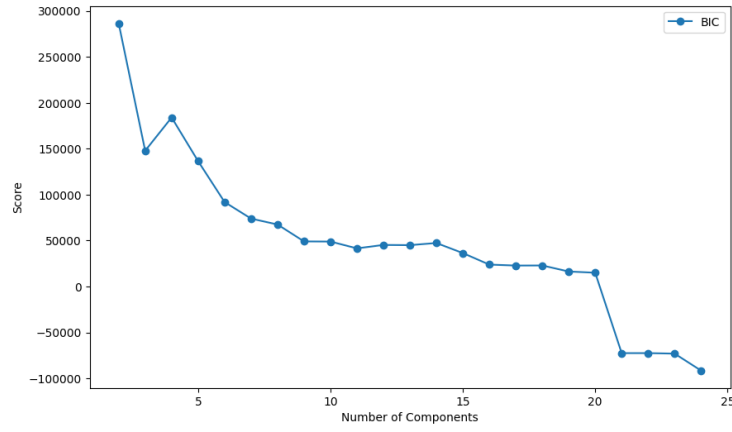


Figure 5.5: BIC using a different number of components

Table 5.4: GMM tuning values with 50th percentile anomaly detection metrics

Comp.	Silhouette	AIC	BIC	LL	ARI	NMI	P	R	F1	AUC	A	Time (s)
2	0.41	286328.97	286428.19	-2.34	0.06	0.03	0.63	0.72	0.67	0.76	0.56	0.19
3	0.27	147639.71	147793.05	-1.21	0.16	0.16	0.60	0.73	0.66	0.78	0.56	0.42
4	0.51	183784.65	183992.10	-1.50	0.14	0.09	0.63	0.76	0.69	0.80	0.56	0.58
5	0.58	136206.19	136467.76	-1.11	0.12	0.08	0.63	0.76	0.69	0.82	0.56	0.30
6	0.50	91626.39	91942.08	-0.75	0.16	0.13	0.63	0.77	0.70	0.83	0.56	0.55
7	0.45	73549.94	73919.74	-0.60	0.16	0.14	0.63	0.77	0.69	0.82	0.56	0.46
8	0.54	67069.24	67493.17	-0.55	0.16	0.13	0.64	0.77	0.70	0.83	0.56	0.56
9	0.49	48608.64	49086.69	-0.40	0.16	0.14	0.64	0.77	0.70	0.82	0.56	0.63
10	0.50	48407.43	48939.59	-0.40	0.16	0.14	0.64	0.77	0.70	0.82	0.56	0.83
11	0.49	40919.07	41505.35	-0.33	0.16	0.15	0.64	0.77	0.70	0.83	0.56	0.96
12	0.54	44605.61	45246.01	-0.36	0.16	0.15	0.63	0.77	0.69	0.83	0.56	0.58
13	0.54	44333.32	45027.84	-0.36	0.16	0.15	0.63	0.77	0.69	0.83	0.56	0.67
14	0.54	46602.47	47351.10	-0.38	0.16	0.15	0.63	0.75	0.69	0.83	0.56	0.72
15	0.50	35591.20	36393.95	-0.29	0.16	0.14	0.64	0.77	0.70	0.84	0.56	0.86
16	0.52	22986.62	23843.49	-0.19	0.16	0.14	0.63	0.77	0.69	0.84	0.56	0.81
17	0.57	21912.06	22823.05	-0.18	0.17	0.15	0.66	0.79	0.71	0.83	0.56	0.87
18	0.58	21938.64	22903.75	-0.18	0.17	0.15	0.66	0.79	0.71	0.83	0.56	0.70
19	0.57	15302.68	16321.90	-0.12	0.17	0.15	0.64	0.76	0.69	0.83	0.56	1.09
20	0.56	13927.41	15000.75	-0.11	0.17	0.15	0.64	0.76	0.69	0.83	0.56	1.08
21	0.55	-73668.23	-72540.77	0.61	0.12	0.15	0.65	0.80	0.72	0.85	0.56	1.07
22	0.55	-73709.30	-72527.73	0.61	0.12	0.15	0.66	0.80	0.72	0.85	0.56	1.20
23	0.53	-74312.75	-73077.05	0.61	0.12	0.15	0.66	0.80	0.72	0.85	0.56	1.16
24	0.53	-92843.27	-91553.45	0.76	0.12	0.15	0.62	0.75	0.68	0.84	0.56	1.30

Table 5.5: GMM metrics for different covariance types with 18 components

Covariance Type	P	R	F1	AUC	A	Time (s)
Full	0.655	0.785	0.714	0.749	0.742	0.85
Tied	0.620	0.755	0.681	0.716	0.709	0.41
Diag	0.671	0.802	0.731	0.764	0.757	0.43
Spherical	0.668	0.791	0.724	0.759	0.753	0.45

5.2.2 K-means

Tuning the K-means model primarily involves identifying the optimal number of clusters for anomaly detection. The tuning consisted of calculating various values to find the optimal number of clusters for anomaly detection. These values are displayed in Table 5.6. Furthermore, the silhouette score and WSSSE with different numbers of clusters, ranging between 2 and 24 is visualised in Figure 5.6 and Figure 5.7.

Observing Figure 5.6 offers insight into the cohesion and separation of the formed clusters with the help of silhouette score. Higher silhouette scores correspond to a model with better-defined clusters. The figure and its corresponding values in Table 5.6 show that as the number of clusters increases, the silhouette score reaches its peak and then starts to decline, suggesting that adding more clusters beyond a certain point does not contribute to better differentiation between data points.

On the other hand, the WSSSE depicted in Figure 5.7, reflects the compactness of the clusters. The WSSSE tends to decrease as the number of clusters increases; however, the rate of this decrease can reveal the optimal clustering choice. A sharp change in the error decrease, or an *elbow* in the graph, can indicate that additional clusters are not meaningfully contributing to the model's performance.

Analysing silhouette scores and WSSSE across different cluster numbers, a balance between cluster separation and compactness can be identified. For instance, looking at both the silhouette and inertia figures, a peak and subsequent plateau or elbow can guide us toward an optimal number of clusters.

In addition to these clustering-specific metrics, the ARI and NMI were considered to assess the model's clustering quality relative to the true labels. The precision, recall, F1-score and AUC in Table 5.6 provide direct indications of the model's ability to classify anomalies correctly, while accuracy provides an overall success rate.

Finally, by inspecting the time taken for each model, it does not seem to take a significant hit. This tells us that the time taken to train on the selected dataset is not bottlenecked by the number of clusters.

With all these insights in mind and using the values in Table 5.6, an optimal peak is identified around 8 – 10 clusters. Because of this, the selected number of clusters was 9.

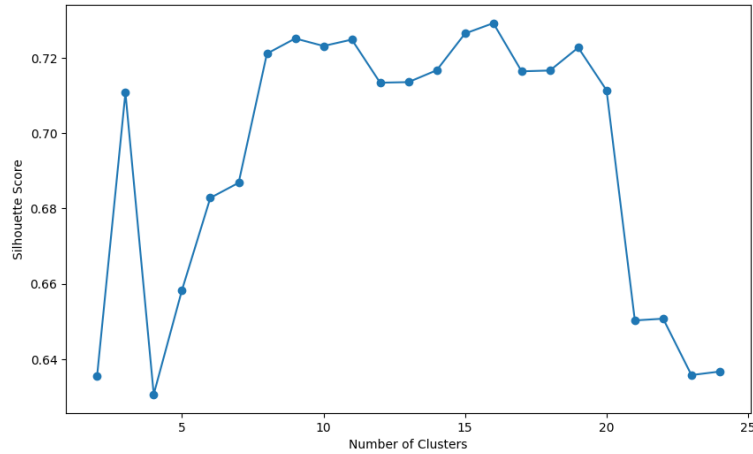


Figure 5.6: Silhouette scores using different numbers of clusters

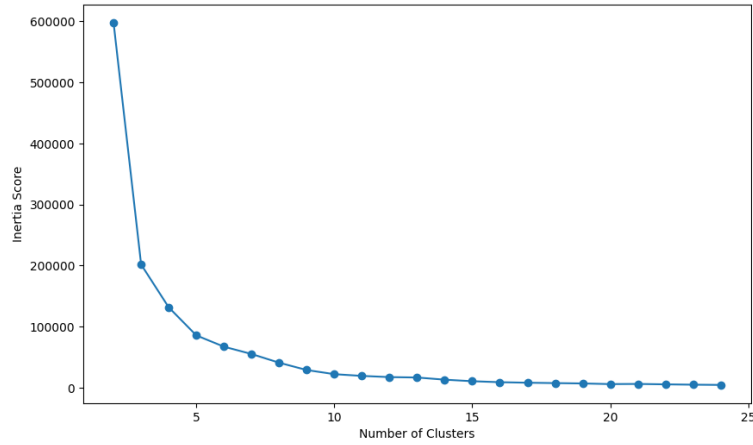


Figure 5.7: Inertia (WSSSE) using different numbers of clusters

Table 5.6: K-means tuning values with 50th percentile anomaly detection metrics

Clusters	Silhouette	WSSSE	ARI	NMI	P	R	F1	AUC	A	Time (s)
2	0.64	598238.23	0.05	0.04	0.52	0.62	0.56	0.58	0.61	0.26
3	0.71	201775.21	0.11	0.06	0.54	0.64	0.59	0.66	0.63	0.24
4	0.63	131632.12	0.10	0.05	0.58	0.67	0.62	0.65	0.66	0.19
5	0.66	85687.88	0.12	0.07	0.50	0.60	0.54	0.69	0.59	0.14
6	0.68	67213.06	0.12	0.07	0.55	0.63	0.59	0.73	0.64	0.16
7	0.69	55201.88	0.12	0.07	0.55	0.63	0.59	0.73	0.64	0.13
8	0.72	40958.00	0.11	0.07	0.56	0.65	0.60	0.77	0.65	0.24
9	0.73	28946.81	0.11	0.07	0.56	0.65	0.60	0.77	0.65	0.18
10	0.72	22194.90	0.11	0.07	0.56	0.64	0.60	0.77	0.65	0.18
11	0.72	19318.17	0.11	0.08	0.55	0.64	0.59	0.76	0.64	0.20
12	0.71	17370.60	0.11	0.08	0.55	0.63	0.58	0.76	0.63	0.25
13	0.71	16714.67	0.11	0.08	0.55	0.63	0.58	0.76	0.63	0.36
14	0.72	13166.42	0.11	0.08	0.55	0.62	0.58	0.76	0.63	0.21
15	0.73	10636.24	0.11	0.09	0.56	0.63	0.59	0.76	0.64	0.18
16	0.73	9025.63	0.11	0.09	0.56	0.63	0.59	0.76	0.64	0.24
17	0.72	8174.58	0.13	0.09	0.52	0.60	0.56	0.77	0.61	0.16
18	0.72	7537.78	0.13	0.09	0.52	0.60	0.56	0.76	0.61	0.31
19	0.72	6961.50	0.13	0.09	0.53	0.61	0.57	0.75	0.62	0.26
20	0.71	5922.45	0.13	0.09	0.53	0.61	0.57	0.75	0.62	0.17
21	0.65	6170.39	0.12	0.12	0.54	0.59	0.56	0.77	0.63	0.42
22	0.65	5512.32	0.12	0.12	0.55	0.59	0.57	0.78	0.63	0.17
23	0.64	4958.96	0.12	0.12	0.55	0.59	0.57	0.78	0.63	0.22
24	0.64	4575.92	0.12	0.11	0.55	0.59	0.56	0.77	0.63	0.25

5.2.3 Isolation Forest

When tuning the IF model, two main parameters were investigated to find an optimal model. The first parameter is the number of estimators, which varies from 100 to 500 in increments of 50 and represents the number of trees in the forest. A higher count typically boosts performance by providing more robustness in the model but also increases the computational demand. The second parameter is the contamination parameter, which adjusts from 0.1 to 0.5 in steps of 0.1, reflecting the expected proportion of outliers within the dataset.

The effects of these parameters on the model's anomaly detection capabilities are detailed in Table 5.7. This table provides a comprehensive evaluation across several metrics, including precision, recall, F1-score, AUC, accuracy, and the time taken for computation. Analysing the

table shows that increasing the contamination level generally enhances the model's recall but often at the expense of reduced precision. This trade-off is a common challenge in anomaly detection, where adjusting the threshold for identifying anomalies can shift the balance between detecting as many true anomalies as possible and avoiding false alarms. Additionally, higher numbers of estimators tend to improve the stability and accuracy of the model, though at higher computational costs. From this evaluation, the optimal IF model is created using a contamination threshold of 0.5, fairly representing the anomaly ratio in the dataset. 300 estimators are selected to have a balanced amount of estimators, even though anomaly detection performance stays the same.

Table 5.7: Isolation Forest performance across different contamination levels and estimators

Contam.	Estimators	P	R	F1	AUC	A	Time (s)
0.10	100	0.84	0.21	0.33	0.59	0.66	0.18
0.10	150	0.85	0.21	0.33	0.59	0.66	0.27
0.10	200	0.85	0.21	0.33	0.59	0.66	0.35
0.10	250	0.85	0.21	0.33	0.59	0.66	0.44
0.10	300	0.85	0.21	0.33	0.59	0.66	0.53
0.10	350	0.85	0.21	0.33	0.59	0.66	0.61
0.10	400	0.85	0.21	0.33	0.59	0.66	0.70
0.10	450	0.85	0.21	0.33	0.59	0.66	0.78
0.10	500	0.85	0.21	0.33	0.59	0.66	0.89
0.20	100	0.74	0.36	0.49	0.64	0.69	0.20
0.20	150	0.73	0.36	0.48	0.63	0.68	0.27
0.20	200	0.73	0.36	0.48	0.63	0.68	0.35
0.20	250	0.73	0.36	0.48	0.63	0.68	0.44
0.20	300	0.73	0.36	0.48	0.63	0.68	0.52
0.20	350	0.73	0.36	0.48	0.63	0.68	0.61
0.20	400	0.73	0.36	0.48	0.63	0.68	0.69
0.20	450	0.73	0.36	0.48	0.63	0.68	0.79
0.20	500	0.73	0.36	0.48	0.63	0.68	0.88
0.30	100	0.71	0.52	0.60	0.69	0.72	0.18
0.30	150	0.72	0.53	0.61	0.69	0.72	0.27
0.30	200	0.72	0.53	0.61	0.69	0.72	0.35
0.30	250	0.72	0.53	0.61	0.69	0.72	0.44
0.30	300	0.73	0.53	0.61	0.70	0.73	0.53
0.30	350	0.72	0.53	0.61	0.69	0.72	0.61
0.30	400	0.72	0.53	0.61	0.69	0.72	0.70
0.30	450	0.72	0.53	0.61	0.69	0.72	0.79
0.30	500	0.72	0.53	0.61	0.69	0.72	0.88
0.40	100	0.70	0.68	0.69	0.74	0.75	0.18
0.40	150	0.69	0.67	0.68	0.73	0.75	0.27
0.40	200	0.70	0.68	0.69	0.74	0.75	0.37
0.40	250	0.70	0.68	0.69	0.74	0.75	0.44
0.40	300	0.70	0.68	0.69	0.74	0.75	0.53
0.40	350	0.70	0.68	0.69	0.74	0.75	0.61
0.40	400	0.70	0.68	0.69	0.74	0.75	0.70
0.40	450	0.70	0.68	0.69	0.74	0.75	0.81
0.40	500	0.70	0.68	0.69	0.74	0.75	0.94
0.50	100	0.67	0.81	0.74	0.77	0.76	0.18
0.50	150	0.66	0.80	0.73	0.76	0.75	0.26
0.50	200	0.66	0.81	0.73	0.76	0.75	0.35
0.50	250	0.66	0.81	0.73	0.76	0.75	0.44
0.50	300	0.67	0.81	0.73	0.76	0.76	0.52
0.50	350	0.66	0.80	0.72	0.76	0.75	0.61
0.50	400	0.66	0.80	0.72	0.76	0.75	0.70
0.50	450	0.66	0.80	0.73	0.76	0.75	0.80
0.50	500	0.66	0.80	0.73	0.76	0.75	0.89

5.2.4 OCSVM

The tuning of the OCSVM model was conducted by assessing various kernel types and subsequently fine-tuning other key parameters, such as ν and γ . Initially, different kernels were evaluated to determine their impact on the model's performance in anomaly detection, as documented in Table 5.8. This preliminary assessment helps identify which kernel has the best balance between sensitivity and specificity for detecting anomalies. This test utilised the baseline parameters for OCSVM, where $\nu = 0.5$ and $\gamma = 'scale'$.

From the table, Radial Basis Function (RBF) kernel demonstrated superior performance in the F1-score, which balances precision and recall, making it a suitable choice for further tuning. On the other hand, kernel *linear*, despite its relatively high precision and similar level of AUC and accuracy, took considerably longer to compute, suggesting less efficiency. The *poly* and *sigmoid* kernels exhibited lower performance across most metrics, indicating their lesser suitability for this specific anomaly detection task. Therefore, *rbf* was the selected kernel for further evaluation.

Subsequent tuning involved adjusting the ν and γ parameters for the selected RBF kernel, aiming to optimise the model's sensitivity to anomalies without compromising computational efficiency. Results of this extensive parameter tuning are detailed in Table 5.9.

ν controls the trade-off between the model's sensitivity to outliers and the number of support vectors used. It essentially sets an upper bound on the fraction of outliers (anomalies) and a lower bound on the number of support vectors. Smaller values of ν increase model sensitivity, allowing more flexibility in separating the classes. Meanwhile, γ influences individual training examples on the decision boundary. In the RBF kernel, γ affects the smoothness of the boundary; a low γ value leads to a smoother boundary, which can be generalised better but might miss some anomalies. Conversely, a high γ value creates a more complex boundary that can capture more anomalies but may also lead to overfitting. The tuning values in Table 5.9 show how adjustments in ν and γ impact precision, recall, F1-score, AUC, accuracy, and computational time.

As ν increases, the model tends to identify more points as anomalies, which increases recall but decreases precision. This adjustment is crucial for balancing the detection of anomalies against the risk of false positives. Changes in γ influence the model's ability to capture the complexity of the data. A proper setting of γ ensures that the model is neither too strict (leading to many false negatives) nor too lenient (resulting in many false positives). The reported times indicate the computational demand, with higher values of ν and lower values of γ generally requiring more processing time due to the increased complexity of the model.

After finding the optimal parameters for ν and γ , the kernels were tested once again with their results showcased in Table 5.10. It can be observed that the kernel *rbf* still outperforms the other kernels in the same way as for the base parameters, suggesting that keeping *rbf* as kernel is the optimal choice.

With the tuning process completed, it was decided that when $\nu = 0.5$ and $\gamma = 'auto'$ using the RBF kernel, the model performed in its optimal state.

Table 5.8: PCA + OCSVM metrics for different kernels where $\nu = 0.5$ and $\gamma = 'scale'$

Kernel	P	R	F1	AUC	A	Time (s)
RBF	0.55	0.66	0.60	0.64	0.64	27.73
Linear	0.68	0.35	0.46	0.62	0.66	124.47
Poly	0.41	0.54	0.46	0.49	0.49	22.33
Sigmoid	0.34	0.40	0.37	0.42	0.42	42.79

Table 5.9: PCA + OCSVM parameter tuning results for RBF kernel

ν	γ	P	R	F1	AUC	A	Time (s)
0.01	scale	0.97	0.02	0.05	0.51	0.60	0.65
0.01	auto	0.97	0.02	0.04	0.51	0.59	0.66
0.01	0.1	0.98	0.02	0.05	0.51	0.60	0.64
0.01	0.01	1.00	0.02	0.05	0.51	0.60	0.58
0.01	0.001	1.00	0.03	0.05	0.51	0.60	0.56
0.01	0.0001	1.00	0.03	0.05	0.51	0.60	0.56
0.05	scale	0.99	0.11	0.21	0.56	0.63	4.08
0.05	auto	0.94	0.13	0.22	0.56	0.63	4.06
0.05	0.1	0.91	0.12	0.21	0.55	0.63	4.12
0.05	0.01	0.92	0.11	0.20	0.55	0.63	3.89
0.05	0.001	0.90	0.11	0.19	0.55	0.62	3.91
0.05	0.0001	0.89	0.11	0.19	0.55	0.62	3.89
0.1	scale	0.86	0.19	0.32	0.59	0.65	8.51
0.1	auto	0.77	0.23	0.35	0.59	0.65	7.71
0.1	0.1	0.91	0.22	0.36	0.60	0.67	7.87
0.1	0.01	0.78	0.16	0.27	0.56	0.63	7.72
0.1	0.001	0.75	0.16	0.27	0.56	0.63	7.83
0.1	0.0001	0.74	0.17	0.27	0.56	0.63	7.82
0.2	scale	0.56	0.31	0.40	0.57	0.61	16.56
0.2	auto	0.80	0.36	0.49	0.65	0.70	14.55
0.2	0.1	0.80	0.35	0.49	0.65	0.69	15.38
0.2	0.01	0.50	0.25	0.33	0.54	0.58	15.00
0.2	0.001	0.57	0.25	0.35	0.56	0.61	15.49
0.2	0.0001	0.49	0.26	0.34	0.53	0.58	15.68
0.3	scale	0.58	0.42	0.49	0.60	0.63	21.74
0.3	auto	0.69	0.47	0.56	0.66	0.69	20.57
0.3	0.1	0.61	0.46	0.52	0.63	0.65	20.97
0.3	0.01	0.50	0.35	0.41	0.55	0.59	22.02
0.3	0.001	0.49	0.34	0.40	0.55	0.58	22.60
0.3	0.0001	0.49	0.34	0.40	0.55	0.58	23.42
0.4	scale	0.58	0.56	0.57	0.64	0.65	25.89
0.4	auto	0.61	0.55	0.58	0.65	0.67	26.73
0.4	0.1	0.58	0.58	0.58	0.65	0.66	25.43
0.4	0.01	0.43	0.44	0.43	0.51	0.53	28.17
0.4	0.001	0.43	0.41	0.42	0.51	0.53	29.26
0.4	0.0001	0.43	0.41	0.42	0.51	0.53	29.93
0.5	scale	0.55	0.66	0.60	0.64	0.64	30.58
0.5	auto	0.59	0.71	0.65	0.68	0.68	29.40
0.5	0.1	0.56	0.68	0.61	0.65	0.64	31.10
0.5	0.01	0.45	0.54	0.49	0.53	0.53	35.00
0.5	0.001	0.40	0.48	0.44	0.49	0.49	36.39
0.5	0.0001	0.40	0.47	0.43	0.49	0.49	37.59

Table 5.10: PCA + OCSVM metrics for different kernels where $\nu = 0.5$ and $\gamma = 'auto'$

Kernel	P	R	F1	AUC	A	Time (s)
RBF	0.59	0.71	0.64	0.68	0.67	24.70
Linear	0.68	0.35	0.46	0.62	0.66	119.13
Poly	0.41	0.54	0.47	0.50	0.49	21.57
Sigmoid	0.44	0.54	0.49	0.53	0.53	46.83

5.3 Tuned Models

All models have seen an improvement after tuning each model. IF has had a significant improvement in reducing the FN ratio, while GMM and K-means has improved slightly. OCSVM has also seen minor improvements but performs worse than random guessing at some thresholds.

The tuned models are configured with the parameters decided using the tuning process covered and can be seen in Listing 4.8. The performance metrics for these models are shown in Table 5.11 and visually compared through ROC curves in Figure 5.8 and confusion matrices in Figure 5.9. The timings slightly differ when tuning due to the variance in running the models. However, these differences are negligible.

Firstly, the tuned GMM model demonstrates enhanced effectiveness in anomaly detection. Its precision of 0.66 suggests a reasonable accuracy in predicting anomalies, while a recall of 0.79 indicates a high capability in capturing actual anomalies. The F1-score of 0.71 reflects a strong balance between precision and recall, reflecting the model's improved efficiency. The AUC score of 0.83 signifies an excellent ability to distinguish between normal requests and anomalies. With an accuracy of 0.74, the model correctly labels the majority of requests. This performance is reinforced by the corresponding confusion matrix, where a notable 80% of anomalies were correctly identified, underscoring the model's reliability after tuning.

Secondly, the tuned K-means show a balanced but moderate enhancement. The precision stands at 0.56, indicating that more than half of its anomaly classifications are likely to be accurate. A recall of 0.65 means that a decent amount of all true anomalies were successfully identified but still have some way to go to be considered acceptable. The model's F1-score, at 0.60, indicates a fair balance between precision and recall, even though it is underwhelming. The AUC score improves to 0.77, suggesting better model discernment. The accuracy score of 0.65 signals that the model accurately labels two-thirds of all requests, which is confirmed by the brighter shades in the confusion matrix for correctly predicting anomalies.

The tuned IF model improved significantly, particularly in the recall score. The precision of 0.67 and recall of 0.80 suggest a robust capability in identifying true anomalies. The model achieves an F1-score of 0.73, indicating an excellent balance between precision and recall. An impressive AUC of 0.83 reflects a strong discriminative ability, verified by the model's accuracy of 0.75. These values suggest that the model is highly effective after tuning. The confusion matrix shows a substantial portion of correct predictions for the anomaly class, reinforcing the model's robustness in distinguishing between classes compared to the baseline performance.

The OCSVM model maintains steady performance, with a precision of 0.59 indicating that it correctly identifies anomalies with fair accuracy. The recall rate of 0.71 reflects its capacity to capture a substantial portion of actual anomalies. The F1-score of 0.64 illustrates a balanced precision-recall performance. The model's AUC score of 0.71 denotes a good, but not excellent, ability to classify between normal and abnormal requests. An overall accuracy of 0.67 implies that two-thirds of all requests are correctly labelled. Despite taking the longest training time among the models, the OCSVM still performs consistently. However, compared to the other models, it shows potential areas for further optimisation, especially in reducing the time complexity.

The performance of the tuned models shows a noticeable improvement compared to the baseline results. In the context of the ROC curves, the models now show a more consistent and desirable performance over the entire threshold range, with all models maintaining a curve above the line of random guessing (AUC scores all being significantly over 0.5). All models' respective confusion matrix now illustrates an overall better performance than the baseline models. The proceeded improvements emphasise the importance of tuning and optimising model parameters to enhance the performance of anomaly detection models.

Finally, by inspecting Table A.2, an updated overview of the clustering metrics for K-means and GMM is presented. Notably, both algorithms show improvements in clustering quality compared to the baseline results. For GMM, the silhouette score has increased to 0.65, indicating better separation between clusters, the ARI and NMI scores have also improved, suggesting enhanced alignment with the true clustering structure. Moreover, the analysis of anomaly information reveals a more balanced distribution of anomalies across clusters, with some clusters achieving perfect separation between normal and abnormal data points (1.0). Similarly, K-means also show improvements in clustering quality, with the silhouette score rising to 0.73. Despite the continued challenge in accurately identifying anomalies, as indicated by the anomaly ratios within clusters, K-means demonstrates enhanced clustering performance compared to the baseline results, even though its anomaly prediction capabilities is not improved as much. The increased silhouette score reflects improved cluster cohesion and separation, highlighting the effectiveness of parameter tuning in enhancing clustering accuracy.

In summary, the tuned table underscores the effectiveness of parameter optimisation in enhancing the clustering performance of both K-means and GMM. The improvements in clustering quality suggest greater alignment with the true data structure, laying a foundation for more accurate anomaly detection and data analysis. Further optimisation and validation may be necessary to fully exploit the potential of these clustering algorithms in practical applications.

Table 5.11: Performance metrics for tuned models

Model	P	R	F1	AUC	A	Time (s)
GMM	0.66	0.79	0.71	0.83	0.74	0.75
K-means	0.56	0.65	0.60	0.77	0.65	0.13
Isolation Forest	0.67	0.80	0.73	0.83	0.75	0.70
OCSVM	0.59	0.71	0.64	0.71	0.67	30.84

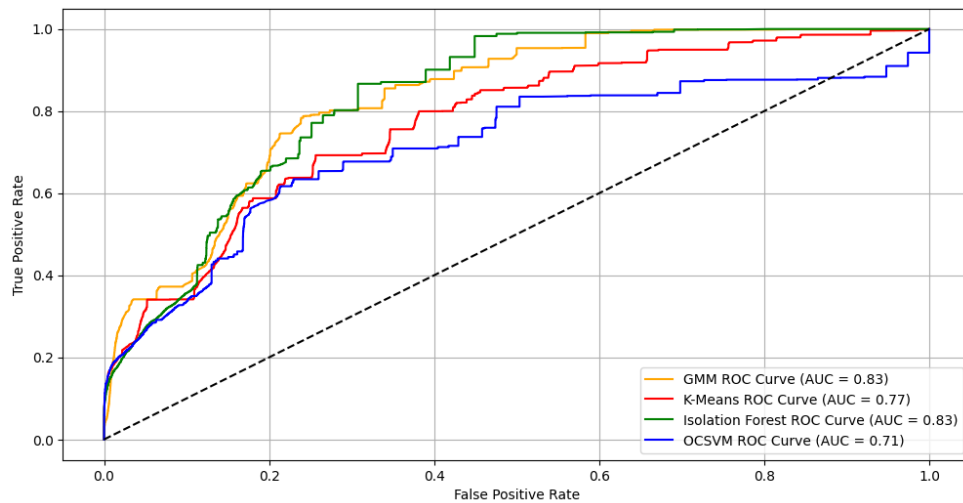


Figure 5.8: Comparison of ROC curves for tuned models

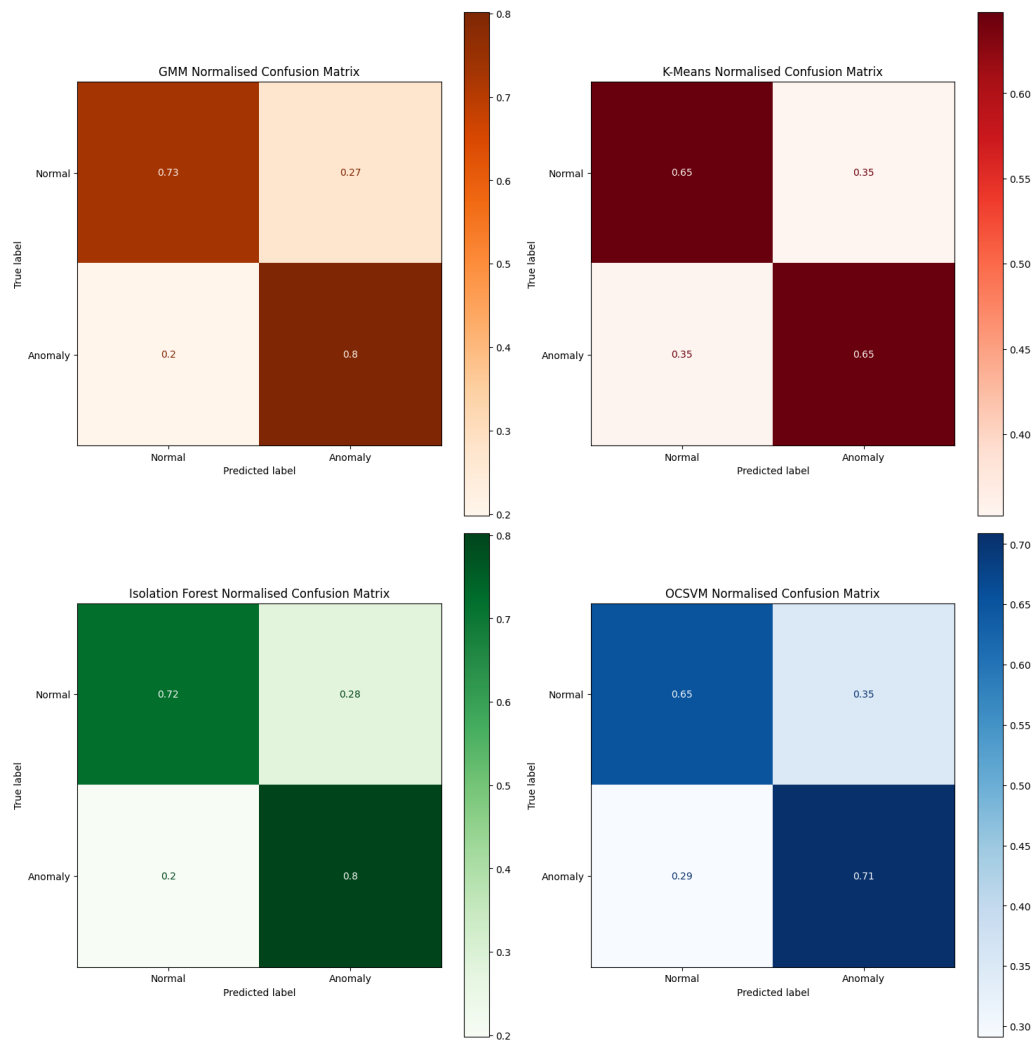


Figure 5.9: Comparison of confusion matrices for tuned models



6 Discussion

This chapter will discuss the results achieved and what it advocates for regarding enhancing API security. Furthermore, the methodology used will be discussed, compared to the literature review performed and if alternative methods could have been executed. The dataset and its limitations will also be brought up. Then, a discussion of the sources will be performed. Finally, the work in a wider context and final discussions regarding API security insights will be discussed to close this chapter.

6.1 Results

This section will discuss the results achieved in Chapter 5 with the used setup described in Section 4.2, including a discussion for each part, highlighting the most interesting findings.

6.1.1 Baseline Models

The best results from Section 5.1 depend on the end goal of using the models. If the goal is to maximise the rate of finding true anomalies, then IF performed best due to its high precision. However, if the goal is to find a balance where the models can both identify true anomalies and not let anomalies slip, i.e. high recall, then GMM performed the best by some margin. It is important to note that even if this performance is acceptable, there are still FPs and FNs in this model as evidenced by Figure 5.2, suggesting that even the best baseline model required improvements. This is clearer for the other models where the presence of FPs and FNs are higher in most areas.

When interpreting the K-means and GMM results, comparing the clustering quality and their respective ability to identify anomalies can seem a bit contradicting. This can make it difficult to determine which model performed better. In the context of this master thesis, the focus is on identifying the anomalies while not flagging normal data as an anomaly and vice versa. In this case, the clustering quality has a lesser meaning. However, it can indicate room for improvement, which can improve the anomaly detection, as noticed by tuning the models. This is especially valuable when investigating the ratio of anomalies per cluster or mixture component.

6.1.2 Tuning the Models

It was noticeable that certain hyperparameters had a dissimilar impact on the model performance than others. One apparent hyperparameter in this case was the number of estimators for the IF model. Here, the precision, recall, F1-score, AUC and accuracy stayed the same, no matter the number of estimators. However, the time taken for training the model increased as the number of estimators increased. The reason why 300 estimators were selected even though it does not affect the model significantly is preference. On the other hand, the *contamination* parameter was the sole parameter affecting the model performance in the evaluation. However, knowing the best value for the *contamination* parameter can become tricky if prior knowledge of the anomaly ratio in the used dataset is lacking. For the OCSVM model, ν greatly impacted the balance between precision and recall, suggesting it had a larger impact compact to γ . However, as ν increased, the impact γ became larger, which makes sense. This is because ν and γ cooperate with the boundary for the model for determining whether an API request is an anomaly or normal.

The selected parameter to tune for each model was selected based on its predicted impact on the model, where some parameters in the end did not change the outcome of the anomaly detection to a significant degree. For K-means and GMM, the number of clusters and mixture components was the key difference. GMM had further tuning with its covariance type tweaked, where it resulted in the *diag* covariance producing the best result, emphasising the importance of testing different setups to find the settings producing optimal performance. Meanwhile, for K-means, no other parameter was tuned as previous studies conducted by e.g. Franzén et al. [51] primarily tuned the number of clusters to find the optimal performance for detecting anomalies. As for OCSVM, the kernel tuning had a similar end goal as the covariance type for GMM but served different purposes. Testing every single combination using every kernel in collaboration with all other combinations (ν and γ) took a long time to perform. Therefore, testing all the covariance types for each combination in Table 5.4 for GMM and all the kernel types for each combination in Table 5.9 was dismissed and only tested using the baseline setup and the tuned setup.

6.1.3 Tuned Models

The results from Section 5.3 tell us that improved models for unsupervised anomaly detection can be achieved by tuning the models carefully. By going back to the mentioned models for the baseline results, GMM and IF, the improvement varies. For GMM and K-means, the improvement is slight. As for IF, the balance between precision and recall is greatly improved, making it a viable option if the end goal is to have a balanced model where both true anomalies (TPs) are found at a high rate while the rate of FP and FN are low.

Compared to the baseline results, the tuned results for K-means and GMM and their ability of identifying anomalies as well as their clustering quality is better aligned. However, since the distance threshold is set to 50%, not all anomalies in the clusters where the anomaly ratio is 1.0 are identified. By inspecting the entire cluster in combination with setting a distance threshold, we can more effectively analyse the results and find more anomalies. In other words, there is not one waterproof way of identifying anomalies in clustering.

The OCSVM model still had trouble where in a few instances it performed worse than randomly guessing whether a data request was an anomaly by inspecting Figure 5.8. Since it does not perform better than any other model in this case renders it an unappealing option out of the evaluated models. This is further proved by its longer training time compared to the rest of the models.

In the end, there is no universally best option for unsupervised anomaly detection [19] since it comes to the need, the characteristics of the specific dataset and what types of anomalies are desired to be found. In this setting and environment, IF and GMM provided the most promising results out of all the models, where GMM can provide more information with its

clusters, such as the anomaly ratio. However, this depends on whether labels are available, which is not expected in a real-life scenario.

6.2 Method

This section will discuss and analyse the methodology used in the thesis, to evaluate different unsupervised anomaly detection approaches in API logs. The most interesting parts will be covered to be criticised, analysed and discussed.

6.2.1 Dataset

As mentioned in Section 1.6, the planned API logs to be used were logs provided by Upsales. These logs contain information about requests to their API and include IP addresses, status codes, timestamps and more. By using their data, direct value could be obtained for Upsales. However, as described in Section 1.6, the nature of how the log data is structured from the logging service Graylog, as well as the absence of domain knowledge and the presence of sensitive information makes the use of the Upsales data much more difficult to utilise. Furthermore, the dataset's size reaches over 100 GB, requiring big data capabilities. This was attempted using services offered by AWS such as Elastic MapReduce (EMR) and Elastic Compute Cloud (EC2), storing the objects in Simple Storage Service (S3). The complexity of using these services in combination with the challenges of the Upsales dataset proved to be too big of a scope for a master thesis and would steer the focus away from the actual aim of the thesis. The issue with the sensitive information in the Upsales data also creates an ethical problem when using the information to train ML models. This is further discussed in Section 6.4.

The dataset used for this thesis, despite its usefulness for API log analysis, does lack some fundamental features to provide complete value to API log analysis. Features such as timestamps, status codes and response time were absent from the dataset. There are other available datasets as presented by Divekar et al. [42], Das et al. [43] and Laskar et al. [34], such as KDDCup1999, NSL-KDD, UNSW-NB15 and iSecurity. These datasets contain anomalies like the CSIC dataset and attacks such as DoS, backdoors and other exploit attacks. However, these datasets provide training and testing material for more network-based anomaly detection and not APIs. For this thesis, the intention was to use a dataset where the API logs structure is suited towards a similar structure Upsales utilises and is the standard API log structure. This could support the future implementation of Upsales in their system. Therefore, using the CSIC dataset was the most appropriate publicly available dataset, excluding the previously discussed Upsales data.

6.2.2 Anomaly Detection Process and Reproducibility

The anomaly detection process used and inspired by Ahmed et al. [19] has proven to be a working methodology process flow when performing unsupervised anomaly detection on API logs. However, the process flow is an overview of the entire process and does not explain the details that go into each step. Therefore, the most interesting parts of the anomaly detection process are discussed to provide further insights.

Data processing

The data processing step is perhaps the most influential step of the entire process. Proper pre-processing is essential to ensure the data is properly represented when training the ML models later. To properly prepare the data and maximise the effectiveness of the anomaly detection models, the data was transformed into numerical features to ensure the models could train on the data.

From the performed methodology in this thesis described in Section 4.2.1, data cleaning was the first performed pre-processing step. This involved handling missing values, which is crucial when preparing the dataset for anomaly detection tasks. This is also a common occurrence in real-life data. There are several reasons as to why handling missing values is important. By filling in missing values, the quality of the dataset increases. This provides enhanced analysis capabilities. The model performance is also increased and not as biased. This allows for an unbiased representation of the patterns in the data. An example of why cleaning the data is important is by looking at the model GMM. To ensure that the means and standard deviations used when training the model are more accurate and appropriately handled, the data is cleaned to make the model reliable. Data cleaning is a common process and should always be performed when dealing with data containing empty fields, missing values or faulty data.

The feature engineering performed in the thesis focuses a lot on the URL, as it is used as a part of the API endpoint definition. When the feature engineering was performed on the URLs, there were keywords defined in the pre-processing stage assigned with weighted severity values, helping define the possible consequences of a specific request to the API. If the request, for instance, contains *password* or *admin* in the URL, this could indicate someone trying to acquire elevated privileges by accessing a user account with many privileges. The weights given to each keyword depended on the severity of the attacks possible by utilising specific keywords, inspired by the severity provided by OWASP in their API vulnerabilities ranking. There are many ways to go regarding feature engineering a dataset, making it as helpful as possible when applying it to anomaly detection models. However, by focusing on feature engineering the focus of this thesis and incorporating severity-weighted keywords, the thesis could aim to enhance the effectiveness of anomaly detection models in identifying and mitigating potential security threats in API environments, specifically via the API endpoints.

As for label encoding, this is a common approach to transform data into a numerical format where the data variance is low and the occurrence of the same data is high. A good example is the HTTP request type, e.g. GET and POST. However, it is important to acknowledge other approaches to encoding common values. One such example is *One-Hot Encoding*. This was not used due to the number of unique values in the dataset, which would significantly increase the complexity of the dataset for the anomaly detection models to interpret. This becomes an even bigger problem when the dataset size increases, impacting the time it takes to detect anomalies if the time for pre-processing the data is a big concern and taken into account. However, One-Hot Encoding could have been a better approach on paper overall, since ordering is not accounted for when labelling into a numerical format, which label encoding does. This is something that could be looked into further for future work.

Finally, the scaling performed was done to ensure that the features have values in the same range, leading to decreased training time and ensuring each feature contributes equally to the result. Would the data not be scaled, then some outliers in the scale could impact the overall performance of detecting anomalies. It is always advised to scale the data to not create biased results, where some instances could skew the entire result of the anomaly detection process.

Model Selection

In the model selection process, multiple models were considered but later omitted to increase the focus on tuning the selected models. One of the considered models was the Local Outlier Factor (LOF) anomaly detection model. According to multiple studies, conducted by e.g. Cheng et al. [23] and Xu et al. [66] suggested that using LOF together with IF led to significant improvements as they compensated each other in time complexity and finding local outliers. This was a considered approach when using IF but was later omitted to solely evaluate IF. An integration using LOF could be evaluated as part of future work.

Combining several models used in this thesis in a streamlined process as performed by Laskar et al. also suggests that using multiple models can enhance the performance rather than using just one. Therefore, one could firstly use a suitable dimensionality reduction technique, such as PCA used in this thesis or perhaps Truncated SVD, depending on the dataset characteristics. Then apply the dimension-reduced feature space to a clustering technique, such as K-means or GMM. Then finally apply e.g. IF to wrap the process up. Another way is to first use IF, then K-means as Laskar et al. did with promising results.

Evaluating the Models

When evaluating the process to determine how the models performed, different metrics were used depending on the model used. However, it was preferred to use metrics that could be compared to other papers if needed. This helps create a consistent comparison between models in different papers. Therefore, metrics such as precision, recall, F1-score, AUC and accuracy were selected for this purpose. However, different datasets are still used so they can not be directly compared but an initial feeling of the performance differences can be determined. One example is comparing the precision, recall and F1-score to the results of He et al. [33]. Here, the results achieved were generally better on the HDFS dataset when compared to the BGL dataset. The results from this master thesis lie somewhere between these two. This further suggests that the dataset used for the evaluation has to be considered.

Another interesting evaluation was the 50% percentile threshold for K-means and GMM. The rationale for the 50th percentile was selected based on the ratio of anomalies in the dataset. This can become difficult to apply in a real-life scenario where we do not know the ratio of anomalies. Therefore, complications can arise in evaluating how well the clustering models perform using other real data where labels are missing. This applies to all models since the evaluating real-life logs lack labelled data. This is a reoccurring challenge in the field of log analysis mentioned by multiple e.g. Ahmed et al. [19]. As a recommendation, where the anomaly ratio is typically less, a percentile around 90 – 95 is a good starting point. Trying out different values around this interval could indicate the true ratio in the dataset.

Reproducibility

The results of this thesis have been collected in such a way that they can be collected repeatedly, given the user has access to the code. However, the specifics of the code are not publicly available due to its potential future implementation in the Upsales system, which could open up future vulnerabilities, should every detail be publicly available.

However, to keep consistency between runs when evaluating the models used, we have to ensure that the results that the models produce are achievable every time. To ensure reproducibility for these models, we need to configure the random state where applicable to set the seed. In the case of this thesis, it was 42. The reason why 42 is selected does not come down to better performance or an even more robust approach, it is just a commonly used random state inside the machine learning community.

6.2.3 Alternative Methods

For this master's thesis, some things could have been done differently, contributing to even better results. These alternatives will be discussed in this section. Furthermore, other approaches not mandatory to this thesis but still relevant to the topic are also discussed to introduce opportunities for related work, such as using unstructured API logs.

Consider More Anomaly Types

There are more attacks in the real world compared to what is present in the dataset. As attacks are the primary data instances that we want to find when using anomaly detection,

introducing more variation could improve the models' capabilities of detecting a wider range of attacks. DDoS attacks, XML External Entity (XXE) attacks, SSRF and login attacks are examples of attack types which could be considered as they can be utilised to target APIs as well. However, it can be challenging to cover more attacks in a real-life dataset since for that to be possible, real instances of such attacks in the chosen system must have occurred previously and recorded and saved in the API log system.

To be able to cover more anomaly types, other applicable datasets have to be used to cover a wider spectrum of anomalies. This is an ongoing challenge in anomaly detection as the availability of real-life data is sparse online. Finding an applicable dataset where domain knowledge of the requests exists, where a wide range of anomaly types, from static to dynamic, exists. This is and will continue to be a difficult obstacle. The main reason for this is based on the sensitive nature of sharing API logs with the public, as it could leak valuable information about people, companies and perhaps even governments.

Another interesting case could be to investigate contextual and collective anomalies on a deeper scale, rather than only point anomalies. Incorporating contextual and collective anomalies into the context of anomaly detection in API logs could greatly enhance the robustness and capabilities of detecting API-specific anomalies. These anomalies could be e.g. connected with time which would be classified as a contextual anomaly, or if a surge of POST requests is made which would be classified as a part of a collective anomaly.

More Tuning

Even more tuning could be done to improve the results further. For K-means and GMM, the *tolerance* could have been tweaked to see whether changing when convergence of the model is reached is reached. This was not done in this thesis as there were no clear motivations behind tweaking this specific parameter from previous studies. The threshold could also have been further investigated to analyse how the percentile threshold impacted the model performances, specifically K-means and GMM in this case. However, since we already knew the ratio of anomalies, this analysis was omitted.

For IF, the number of max samples could have been tuned to see the effect on the model performance. But as with K-means and GMM regarding the *tolerance*, there was no clear motivation behind tweaking it. However, this parameter is based on the estimators used in the ensembling technique, which can significantly impact the performance, rather than the number of estimators investigated in this thesis.

Finally, for OCSVM, the *tolerance* parameter could have also been tuned here but has the same motivation as for K-means and GMM regarding this parameter. Other parameters that could have been looked at were the *degree* parameter and the *coef0* parameter. These parameters matter when using either the *poly* kernel or the *sigmoid* kernel sometimes. These were omitted due to the dominant performance of the *rbf* kernel. However, using various configurations of the *degree* and *coef0* parameter together with e.g. *poly* kernel could perhaps have performed better. This can be saved for future work. However, there were no papers found stating this.

The time limitation led to the decision to focus on fewer parameters but investigate them further as compensation. Furthermore, a delimitation had to be set to not acquire too much information, which could be considered noise in the results. Also, this opens up further opportunities for future work.

Using Unstructured Logs

In many cases, generated logs from a web application are typically in an unstructured format. When dealing with unstructured raw logs as input data, log parsing becomes necessary to structure the logs for further processing. This procedure will not be required for this master thesis as the logs from the dataset are already structured. However, it is important to

acknowledge that log parsing applications such as Monilog [40], Distributed Log Analysis Framework (DILAF) [48] and projects from *Logpai* such as *loglizer* and *logparser* [37, 67] are commonly used for this purpose if required. These tools streamline the process of log parsing by converting unstructured logs into a structured format. Figure 6.1 [68] visualises the process of log parsing, which is essential for effective anomaly detection when the collected data is unstructured. This step was unnecessary for this thesis as the log service Graylog does this process for us already. However, it is important to know what has to be performed if unstructured logs are the only available ones. In the case of Upsales, should the Graylog service be switched out and unstructured logs become the new normal, knowing how to deal with unstructured logs is good to have in mind.

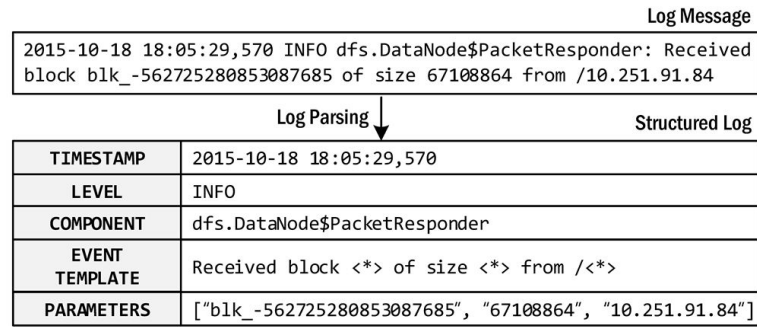


Figure 6.1: Log parsing example

Supervised Anomaly Detection

Another method one could utilise which has proven to perform more efficiently in contrast to unsupervised approaches and produce better results is using supervised ML approaches [33], as presented in Section 3.2. These approaches typically produce better precision, recall and F1-score. On the other hand, unsupervised is not based on knowledge provided by the user. This goes against supervised anomaly detection since it becomes impossible to detect zero-day vulnerabilities [19]. Since the requests coming in and logs generated as a result of the request do not have a label stating whether it is a normal request or anomaly, it is impossible to utilise supervised anomaly detection in the case of Upsales.

6.3 Source Criticism

The literature study consisted of a three-step methodology. Firstly, keywords were identified. Secondly, search strings were constructed and finally inclusion and exclusion criteria were set. The focus was to gather information about the overall process of using anomaly detection in API logs where possible, otherwise logs in general. Furthermore, the focus was gathering literature containing information regarding unsupervised anomaly detection. The findings from the search strings used are presented in Section 3.1 and are available in the bibliography.

All papers included in the final collection are either published articles, part of published conference papers or other types of proceedings. One main concern about the majority of articles selected is that they do not target specifically API logs due to the limited availability of such articles. For instance, Sowmya et al. [30] focuses on APIs, while other articles such as Xu et al. [49] uses more general logs from software systems or similar. The number of papers aiming for logs gathered from networks and other systems outweighs the number of papers specifically experimenting with API logs. This implies that the exact process can not be converted to this thesis. However, inspiration can still be gathered like this thesis's anomaly detection process was inspired by Ahmed et al [19].

6.4 The work in a wider context

There has been plenty of discussion regarding the rise of AI and ML. One common problem brought up is the privacy risks and their ethical consequences.

When it comes to ML and specifically improving security using ML, it is important to acknowledge significant privacy considerations that must be considered. When using real-world data for anomaly detection in API logs, it is crucial to understand that sensitive information in API logs exists, creating privacy issues about whether using the data for ML purposes is correct. There are concerns about whether the data used for training can be leaked by attackers who get access to the ML model. Furthermore, if a model is overfitted on a dataset, then sensitive data could be obtained via an inference attack. Shokri et al. [69] discussed a privacy breach where an adversary could from the output of the model infer sensitive values. Shokri et al. further discuss *Danielius desideratum* where it is stated that the model should not reveal more about the input to which it is applied than would have been known about this input without applying the model. This is however not achievable by any useful model, which has to be considered when deciding to implement ML to enhance API security.

Saltz et al. [70] discussed an issue with ML courses perhaps not discussing ethics enough. This can lead to a misunderstanding of the potential of ML, as it is not only positive. However, findings from Saltz et al. tell us that ethics is thoroughly discussed in ML courses. This is an important factor to further enlighten interested students into the risks of ML, especially applying it to sensitive areas such as security logs or medicine.

From a societal and economic perspective, using an in-house solution can decrease costs as mentioned in the introduction to the thesis. This can help a company rely less on third parties such as the penetration testers or consultants analysing the API logs for Upsales. However, the volume of data available can still lead to significant costs for running the models continuously. A decision has to be carefully considered from one company to another.

6.5 Final Discussions

This master thesis has evaluated multiple models applicable to unsupervised anomaly detection. Furthermore, a discussion of the methodology used, the results and the sources used as knowledge has been performed. This leads to an enhanced view of applying anomaly detection to API security.

APIs today are huge systems that expand way larger than the dataset used in this thesis. Multiple previous studies investigated the usage of real-time anomaly detection using frameworks such as Apache Spark, allowing for anomaly detection in big data. This thesis has been limited to a smaller dataset size. From the results gathered and the previous studies used, it can be determined that applying unsupervised anomaly detection on API logs to enhance API security is indeed an opportunity worth exploring, especially for companies like Upsales where the importance of security and data integrity is crucial. However, due to the expanding scale of APIs, the computational capabilities have to be available to perform unsupervised anomaly detection on larger datasets. This can be achieved by using services such as AWS, offering storage capabilities using S3 and computational services such as EC2 and EMR, depending on the needs for the specific application.

Even though it is possible to apply unsupervised anomaly detection in an API context, it is important to acknowledge the challenges that come with it. As mentioned, APIs are huge systems and will continue growing in size. This will require more computational power to have the anomaly detection models under control. Using effective anomaly detection models, such as IF evaluated in this thesis, scaling capabilities are possible. However, as Laskar et al. [34] discussed, IF is not a well-supported model in Apache Spark and has issues and difficulties dealing with big data scenarios, as Upsales might face. This is a limitation for multiple which has to be accounted for. Selecting a proper system that can be used for big

data analysis and support models appropriate for anomaly detection is a process needed to be carefully performed.

7

Conclusions and Future Work

This chapter concludes this master thesis and summarises the research questions and the respective answers. Furthermore, the future work available after this master thesis is introduced to cover the possibilities moving forward.

7.1 Research Questions

1. **What is the process of implementing anomaly detection in API logs, and what are the applicable anomaly detection models for API log analysis?**

Implementing anomaly detection in API logs contains four major phases: the preparation phase, the anomaly detection phase, the output phase and the evaluation phase. The preparation phase involves gathering the input of the data and preprocessing the log data so it can be applied to the anomaly detection models. The anomaly detection phase involves applying the selected models to the data. The output phase involves receiving output from the models, telling us how well the anomaly detection went. Finally, the evaluation phase is where the results are interpreted and analysed, deciding whether tuning of the model is required or if the results are satisfactory. There is no definitive way of approaching anomaly detection in API logs. However, following a proven process increases the chances of success.

As for the applicable models to unsupervised anomaly detection, several models can be applied. The ones used in this thesis, K-means, GMM, IF, OCSVM and PCA can be applied to unsupervised anomaly detection with varying results. Other applicable unsupervised models such as Truncated SVD and LOF can also be applied.

2. **How effective, in terms of performance and time, are the different anomaly detection models, specifically in detecting abnormal activity towards API endpoints?**

From a baseline perspective, the GMM model proved to have the best balance of precision and recall. IF managed to identify the most true anomalies (high TP rate and low FP rate) out of all the models, at the expense of not identifying anomalies often enough (high FN rate). Meanwhile, K-means is the quickest model, only taking 0.12 seconds, with IF close to matching it. The OCSVM model provided decent performance but at the expense of taking a long time to train.

As for the tuned models, GMM and IF still stand out, achieving good promising results with an F1-score of 0.71 and 0.73 respectively and similar AUC and accuracy levels as well as taking around the same time to train. Meanwhile K-means and OCSVM fall a bit short where OCSVM still have issues with training as fast as the others, with K-means still being as fast as its baseline variant.

3. What insights can be gained using unsupervised anomaly detection within API security, focusing on abnormal activities towards API endpoints?

Using unsupervised anomaly detection to enhance API security is a possibility, considering the results achieved in this master thesis. The models were able to identify a decent range of API attacks, such as XSS attacks and SQL injections as well as other static, dynamic and unintentional illegal behaviour. Different models could detect more anomalies than the other, suggesting that selecting the proper model to enhance API security is pivotal. Proper pre-processing of the API endpoint information is crucial to maximising the anomaly detection results. However, to achieve acceptable results where the dataset size is much bigger, big data capabilities would be required to enhance the API security. One could use Apache Spark to process large data in a distributed way, S3 for storing the data in services offered by the likes of AWS and running the Spark jobs using AWS services such as EMR or EC2 or a combination of both.

7.2 Future Work

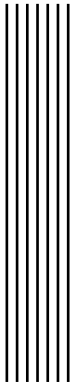
The first major future work is applying the models on Upsales API logs, where more pre-processing is required compared to what was done in this master thesis. Using Upsales API logs, more fundamental API log data can be introduced, such as timestamps and status codes. This data type is vital to find anomalies tied to e.g. time-specific activity and a deviation of status codes.

Should Upsales API logs not be available, the best alternative is to introduce other API logs at least containing timestamps such that time-specific anomalies exist in the dataset, challenging the tested models in this thesis to find time-related anomalies. However, for this to work properly, the timestamp information needs to be pre-processed accordingly, which has not been considered in this thesis.

If more analysis of the dataset used in this thesis is required, one could e.g. analyse the usage of the *poly* kernel together with different configurations of the *degree* and *coef0* parameter. As presented in Section 6.2, tuning more parameters per model can also be performed to experiment with whether even better configurations can be found for this dataset. Furthermore, investigating an interval around the best optimal parameters could also be performed as future work to see whether adjusting it slightly could affect the performance of the anomaly detection models.

One-hot encoding could be compared to label encoding for the pre-processing stage to see whether it performs better than label encoding. Since by design, it should perform better but depends on the nature of the dataset characteristics. This is something that can be investigated further.

Finally, more models could be evaluated such as using other dimensionality reduction techniques such as Truncated SVD as well as combining IF with LOF as discussed or perhaps combining models used in this thesis, e.g. using a clustering technique together with IF, as performed by Laskar et al. [34].



Bibliography

- [1] OWASP. *OWASP Top 10 API Security Risks – 2023*. 2023. URL: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/> (visited on 11/13/2023).
- [2] IBM. *Cost of a Data Breach Report 2023*. 2023. URL: <https://www.ibm.com/reports/data-breach> (visited on 11/11/2023).
- [3] OWASP. *OWASP Risk Rating Methodology*. 2023. URL: https://owasp.org/www-community/OWASP_Risk_Rating_Methodology (visited on 11/13/2023).
- [4] IBM. *What is penetration testing?* 2023. URL: <https://www.ibm.com/topics/penetration-testing> (visited on 11/12/2023).
- [5] Pradyun Gedam. *Welcome to PyOD documentation!* 2023. URL: <https://pyod.readthedocs.io/en/latest/> (visited on 11/29/2023).
- [6] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.
- [7] Apache Spark. *MLlib is Apache Spark’s scalable machine learning library*. 2024. URL: <https://spark.apache.org/mllib/> (visited on 03/08/2024).
- [8] Gaspard Baye, Fatima Hussain, Alma Oracevic, Rasheed Hussain, and S.M. Ahsan Kazmi. “API Security in Large Enterprises: Leveraging Machine Learning for Anomaly Detection”. In: *2021 International Symposium on Networks, Computers and Communications (ISNCC)*. 2021, pp. 1–6. DOI: 10.1109/ISNCC52172.2021.9615638.
- [9] Upsales Nordic AB. *Upsales*. 2023. URL: <https://www.upsales.com/sv/> (visited on 11/11/2023).
- [10] APIToolkit. *API Logs: Your First Line of Defense in Cybersecurity*. 2023. URL: <https://apitoolkit.io/blog/api-logs-in-cybersecurity/> (visited on 04/09/2024).
- [11] OWASP. *About the OWASP Foundation*. 2023. URL: <https://owasp.org/about/> (visited on 11/23/2023).
- [12] OWASP. *OWASP Top Ten*. 2023. URL: <https://owasp.org/www-project-top-ten/> (visited on 11/23/2023).

- [13] Fatima Hussain, Rasheed Hussain, Brett Noye, and Salah Sharieh. "Enterprise API Security and GDPR Compliance: Design and Implementation Perspective". In: *IT Professional* 22.5 (2020), pp. 81–89. DOI: 10.1109/MITP.2020.2973852.
- [14] Eoftedal. *Server Side Request Forgery*. 2023. URL: https://owasp.org/www-community/attacks/Server_Side_Request_Forgery (visited on 02/20/2024).
- [15] Monowar Hussain Bhuyan, Dhruba Kumar Bhattacharyya, and Jugal Kalita. "Network Anomaly Detection: Methods, Systems and Tools". In: *IEEE Communications Surveys & Tutorials* 16.1 (2014), pp. 303–336. DOI: 10.1109/SURV.2013.052213.00046.
- [16] Animesh Patcha and Jung Min Park. "An overview of anomaly detection techniques: Existing solutions and latest technological trends". In: *Computer Networks* 51.12 (2007), pp. 3448–3470. ISSN: 1389-1286. DOI: 10.1016/j.comnet.2007.02.001. URL: <https://www.sciencedirect.com/science/article/pii/S138912860700062X>.
- [17] Patrick Schneider and Fatos Xhafa. *Anomaly detection and complex event processing over IOT data streams with application to eHealth and patient data monitoring*. Academic Press, an imprint of Elsevier, 2022. DOI: 10.1016/C2020-0-00589-X.
- [18] Sherenaz W Al-Haj Baddar, Alessio Merlo, and Mauro Migliardi. "Anomaly detection in computer networks: A state-of-the-art review." In: *J. Wirel. Mob. Networks Ubiquitous Comput. Dependable Appl.* 5.4 (2014), pp. 29–64.
- [19] Mohiuddin Ahmed, Abdun Naser, and Jiankun Hu. "A survey of network anomaly detection techniques". In: *Journal of Network and Computer Applications* 60 (2016), pp. 19–31. ISSN: 1084-8045. DOI: 10.1016/j.jnca.2015.11.016. URL: <https://www.sciencedirect.com/science/article/pii/S1084804515002891>.
- [20] Google Cloud. *What is unsupervised learning?* 2023. URL: <https://cloud.google.com/discover/what-is-unsupervised-learning#:~:text=Unsupervised%20learning%20in%20artificial%20intelligence,any%20explicit%20guidance%20or%20instruction.> (visited on 11/24/2023).
- [21] Carmen Sánchez-Zas, Xavier Larriva-Novo, Víctor A Villagrà, Mario Sanz Rodrigo, and José Ignacio Moreno. "Design and Evaluation of Unsupervised Machine Learning Models for Anomaly Detection in Streaming Cybersecurity Logs". In: *Mathematics* 10.21 (2022), p. 4043.
- [22] Mei-Ling Shyu, Shu-Ching Chen, Kanoksri Sarinnapakorn, and LiWu Chang. "A novel anomaly detection scheme based on principal component classifier". In: *Proceedings of the IEEE foundations and new directions of data mining workshop*. IEEE Press. 2003, pp. 172–179.
- [23] Zhangyu Cheng, Chengming Zou, and Jianwei Dong. "Outlier detection using isolation forest and local outlier factor". In: *Proceedings of the conference on research in adaptive and convergent systems*. 2019, pp. 161–168.
- [24] Kai Guo, Datong Liu, Yu Peng, and Xiyuan Peng. "Data-Driven Anomaly Detection Using OCSVM with Boundary Optimization". In: *2018 Prognostics and System Health Management Conference (PHM-Chongqing)*. 2018, pp. 244–248. DOI: 10.1109/PHM-Chongqing.2018.00048.
- [25] Kun Yang, Samory Kpotufe, and Nick Feamster. *An Efficient One-Class SVM for Anomaly Detection in the Internet of Things*. 2021. arXiv: 2104.11146 [cs.NI].
- [26] GeeksForGeeks. *Semi-Supervised Learning in ML*. 2024. URL: <https://www.geeksforgeeks.org/ml-semi-supervised-learning/> (visited on 01/18/2024).

-
- [27] Google Cloud. *What is supervised learning?* 2023. URL: [https://cloud.google.com/discover/what-is-supervised-learning#:~:text=Supervised%20learning%20is%20a%20category,the%20input%20and%20the%20outputs.\(visited on 11/24/2023\).](https://cloud.google.com/discover/what-is-supervised-learning#:~:text=Supervised%20learning%20is%20a%20category,the%20input%20and%20the%20outputs.(visited%20on%2011%2F24%2F2023).)
 - [28] Khan Academy. *Z-scores review*. 2024. URL: [https://www.khanacademy.org/math/statistics-probability/modeling-distributions-of-data/z-scores/a/z-scores-review#:~:text=A%20z%20score%20measures%20exactly,z%20%3D%20x%20%E2%88%92%20%E2%80%8D\(visited on 02/01/2024\).](https://www.khanacademy.org/math/statistics-probability/modeling-distributions-of-data/z-scores/a/z-scores-review#:~:text=A%20z%20score%20measures%20exactly,z%20%3D%20x%20%E2%88%92%20%E2%80%8D(visited%20on%2002%2F01%2F2024).)
 - [29] S. Gopal Krishna Patro and Kishore Kumar Sahu. *Normalization: A Preprocessing Stage*. 2015. arXiv: 1503.06462 [cs.OH].
 - [30] M. Sowmya, Ankith J. Rai, V. Spoorthi, Md. Irfan, Prasad B. Honnavalli, and S. Nagasundari. "API Traffic Anomaly Detection in Microservice Architecture". In: *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing Workshops (CCGridW)*. 2023, pp. 206–213. DOI: 10.1109/CCGridW59191.2023.00044.
 - [31] Rockwell Schrock. *The Boolean Machine*. 2024. URL: <https://www.rockwellschrock.com/rbs3k/boolean/> (visited on 01/30/2024).
 - [32] R. Kumari, Sheetanshu, M. K. Singh, R. Jha, and N.K. Singh. "Anomaly detection in network traffic using K-mean clustering". In: *2016 3rd International Conference on Recent Advances in Information Technology (RAIT)*. 2016, pp. 387–393. DOI: 10.1109/RAIT.2016.7507933.
 - [33] Shilin He, Jieming Zhu, Pinjia He, and Michael R. Lyu. "Experience Report: System Log Analysis for Anomaly Detection". In: *2016 IEEE 27th International Symposium on Software Reliability Engineering (ISSRE)*. 2016, pp. 207–218. DOI: 10.1109/ISSRE.2016.21.
 - [34] Md. Tahmid Rahman Laskar, Jimmy Xiangji Huang, Vladan Smetana, Chris Stewart, Kees Pouw, Aijun An, Stephen Chan, and Lei Liu. "Extending Isolation Forest for Anomaly Detection in Big Data via K-Means". In: *ACM Trans. Cyber-Phys. Syst.* 5.4 (2021). ISSN: 2378-962X. DOI: 10.1145/3460976. URL: <https://doi.org/10.1145/3460976>.
 - [35] Yifan Guo, Tianxi Ji, Qianlong Wang, Lixing Yu, Geyong Min, and Pan Li. "Unsupervised Anomaly Detection in IoT Systems for Smart Cities". In: *IEEE Transactions on Network Science and Engineering* 7.4 (2020), pp. 2231–2242. DOI: 10.1109/TNSE.2020.3027543.
 - [36] Mostafa Farshchi. "Anomaly detection using logs and metrics analysis for system application operations". In: (2018).
 - [37] Tapio Anttila. "Developing a log file analysis tool: A machine learning approach for anomaly detection". MA thesis. 2020.
 - [38] Merve Astekin, Selim Özcan, and Hasan Sözer. "Incremental Analysis of Large-Scale System Logs for Anomaly Detection". In: *2019 IEEE International Conference on Big Data (Big Data)*. 2019, pp. 2119–2127. DOI: 10.1109/BigData47090.2019.9006593.
 - [39] Deepali Arun Bhanage, Ambika Vishal Pawar, and Ketan Kotecha. "IT Infrastructure Anomaly Detection and Failure Handling: A Systematic Literature Review Focusing on Datasets, Log Preprocessing, Machine & Deep Learning Approaches and Automated Tool". In: *IEEE Access* 9 (2021), pp. 156392–156421. DOI: 10.1109/ACCESS.2021.3128283.
 - [40] Arthur Vervaet. "MoniLog: An Automated Log-Based Anomaly Detection System for Cloud Computing Infrastructures". In: *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. 2021, pp. 2739–2743. DOI: 10.1109/ICDE51399.2021.00317.

- [41] Riyaz Ahamed Ariyaluran Habeeb, Fariza Nasaruddin, Abdullah Gani, Mohamed Ahzam Amanullah, Ibrahim Abaker Targio Hashem, Ejaz Ahmed, and Muhammad Imran. "Clustering-based real-time anomaly detection—A breakthrough in big data technologies". In: *Transactions on Emerging Telecommunications Technologies* 33.8 (2022), e3647.
- [42] Abhishek Divekar, Meet Parekh, Vaibhav Savla, Rudra Mishra, and Mahesh Shirole. "Benchmarking datasets for anomaly-based network intrusion detection: KDD CUP 99 alternatives". In: *2018 IEEE 3rd international conference on computing, communication and security (ICCCS)*. IEEE. 2018, pp. 1–8.
- [43] Kaustav Das and Jeff Schneider. "Detecting anomalous records in categorical datasets". In: *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*. 2007, pp. 220–229.
- [44] Merve Astekin, Harun Zengin, and Hasan Sözer. "Evaluation of distributed machine learning algorithms for anomaly detection from large-scale system logs: A case study". In: *2018 IEEE international conference on big data (big data)*. IEEE. 2018, pp. 2071–2077.
- [45] Sohaila Eltanbouly, May Bashendy, Noora AlNaimi, Zina Chkirbene, and Aiman Erbad. "Machine learning techniques for network anomaly detection: A survey". In: *2020 IEEE International Conference on Informatics, IoT, and Enabling Technologies (ICIoT)*. IEEE. 2020, pp. 156–162.
- [46] Antti Juvonen, Tuomo Sipola, and Timo Hämäläinen. "Online anomaly detection using dimensionality reduction techniques for HTTP log analysis". In: *Computer Networks* 91 (2015), pp. 46–56.
- [47] Max Landauer, Florian Skopik, Markus Wurzenberger, and Andreas Rauber. "System log clustering approaches for cyber security applications: A survey". In: *Computers & Security* 92 (2020), p. 101739. ISSN: 0167-4048. DOI: 10.1016/j.cose.2020.101739.
- [48] Merve Astekin, Harun Zengin, and Hasan Sözer. "DILAF: A framework for distributed analysis of large-scale system logs for anomaly detection". In: *Software: Practice and Experience* 49.2 (2019), pp. 153–170. DOI: 10.1002/spe.2653.
- [49] Wei Xu, Ling Huang, Armando Fox, David Patterson, and Michael I. Jordan. "Detecting large-scale system problems by mining console logs". In: *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles*. SOSP '09. Big Sky, Montana, USA: Association for Computing Machinery, 2009, pp. 117–132. ISBN: 9781605587523. DOI: 10.1145/1629575.1629587.
- [50] Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R. Lyu. *Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics*. 2023. arXiv: 2008.06448 [cs.SE].
- [51] Måns Fredriksson Franzén and Nils Tyrén. "Anomaly Detection for Automated Security Log Analysis: Comparison of Existing Techniques and Tools". PhD thesis. 2021. URL: <https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-177728>.
- [52] Elasticsearch B.V. *What is Elasticsearch?* 2023. URL: <https://www.elastic.co/guide/en/elasticsearch/reference/current/elasticsearch-intro.html> (visited on 11/28/2023).
- [53] Firebase. *Cloud Firestore*. 2023. URL: <https://firebase.google.com/docs/firestore> (visited on 12/10/2023).
- [54] Mohammed Ansari. "Anomaly Detection Using Unsupervised Learning Technique". MA thesis. Itä-Suomen yliopisto, 2021.

- [55] Aidong Xu, Lin Chen, Yixin Jiang, Huahui Lv, Hang Yang, and Bo Li. "Finding Gold in the Sand: Identifying Anomaly Indicators Through Huge Amount Security Logs". In: *2020 IEEE 6th Intl Conference on Big Data Security on Cloud (BigDataSecurity), IEEE Intl Conference on High Performance and Smart Computing, (HPSC) and IEEE Intl Conference on Intelligent Data and Security (IDS)*. 2020, pp. 140–144. DOI: 10.1109/BigDataSecurity-HPSC-IDS49724.2020.00034.
- [56] Scott Lupton, Lena Yu, Hironori Washizaki, Nobukazu Yoshioka, and Yoshiaki Fukazawa. "Assessment of Real-World Incident Detection Through a Component-Based Online Log Anomaly Detection Pipeline Framework". In: *2023 10th International Conference on Dependable Systems and Their Applications (DSA)*. 2023, pp. 477–478. DOI: 10.1109/DSA59317.2023.00066.
- [57] Ravi Guntur. *API security: Access behaviour anomaly dataset*. 2021. URL: https://www.kaggle.com/datasets/tangodelta/api-access-behaviour-anomaly-dataset?select=remaining_behavior_ext.csv (visited on 04/02/2024).
- [58] Carmen Torrano Giménez, Alejandro Pérez Villegas, and Gonzalo Álvarez Marañón. *HTTP DATASET CSIC 2010*. 2012. URL: <https://www.tic.itefi.csic.es/dataset/> (visited on 04/02/2024).
- [59] I. Spangler. *CSIC 2010 Web Application Attacks*. 2019. URL: <https://www.kaggle.com/datasets/ispangler/csic-2010-web-application-attacks/data> (visited on 04/02/2024).
- [60] Oumaima Chakir, Abdeslam Rehaïmi, Yassine Sadqi, El Arbi Abdellaoui Alaoui, Moez Krichen, Gurjot Singh Gaba, and Andrei Gurtov. "An empirical assessment of ensemble methods and traditional machine learning techniques for web-based attack detection in industry 5.0". In: *Journal of King Saud University - Computer and Information Sciences* 35.3 (2023), pp. 103–119. ISSN: 1319-1578. DOI: 10.1016/j.jksuci.2023.02.009.
- [61] Tyler Lanigan, Sebastian Raschka, and Arturo Amor. *Importance of Feature Scaling*. 2024. URL: https://scikit-learn.org/stable/auto_examples/preprocessing/plot_scaling_importance.html#:~:text=A%20clear%20difference%20in%20prediction,vastly%20outperforms%20the%20unscaled%20version. (visited on 04/21/2024).
- [62] Suleman Khan, Joakim Thorn, Alex Wahlgren, and Andrei Gurtov. "Intrusion Detection in Automatic Dependent Surveillance-Broadcast (ADS-B) with Machine Learning". In: *2021 IEEE/AIAA 40th Digital Avionics Systems Conference (DASC)*. 2021, pp. 1–10. DOI: 10.1109/DASC52595.2021.9594431.
- [63] Wahid Salman Al Farizi, Indriana Hidayah, and Muhammad Nur Rizal. "Isolation Forest Based Anomaly Detection: A Systematic Literature Review". In: *2021 8th International Conference on Information Technology, Computer and Electrical Engineering (ICI-TACEE)*. 2021, pp. 118–122. DOI: 10.1109/ICITACEE53184.2021.9617498.
- [64] Feng Zhou and Hua Qu. "A GMM-Based Anomaly IP Detection Model from Security Logs". In: *Smart Computing and Communication*. Ed. by Meikang Qiu. Cham: Springer International Publishing, 2021, pp. 97–105.
- [65] Gopinath Muruti, Fiza Abdul Rahim, and Zul-Azri bin Ibrahim. "A Survey on Anomalies Detection Techniques and Measurement Methods". In: *2018 IEEE Conference on Application, Information and Network Security (AINS)*. 2018, pp. 81–86. DOI: 10.1109/AINS.2018.8631436.
- [66] Dong Xu, Yanjun Wang, Yulong Meng, and Ziyang Zhang. "An Improved Data Anomaly Detection Method Based on Isolation Forest". In: *2017 10th International Symposium on Computational Intelligence and Design (ISCID)*. Vol. 2. 2017, pp. 287–291. DOI: 10.1109/ISCID.2017.202.

-
- [67] Pinjia He, Zhujiem, JinYang88, Jinyang LIU, Siyuexi, Junjielong Xu, Shilin He, Joe-HitHard, Joseph Hit Hard, rustamtemirov, Rustam Temirov, gaiusyu, Siyu Yu (Youth Yu), thomasryck, IsuruBoyagane15, and Isuru Boyagane. *LOGPAI*. 2024. URL: <https://github.com/logpai> (visited on 02/14/2024).
 - [68] Pinjia He, Zhujiem, JinYang88, Jinyang LIU, Siyuexi, Junjielong Xu, Shilin He, Joe-HitHard, Joseph Hit Hard, rustamtemirov, Rustam Temirov, gaiusyu, Siyu Yu (Youth Yu), thomasryck, IsuruBoyagane15, and Isuru Boyagane. *Logpai LogParser*. 2024. URL: <https://github.com/logpai/logparser/blob/main/README.md> (visited on 02/08/2024).
 - [69] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. "Membership Inference Attacks Against Machine Learning Models". In: *2017 IEEE Symposium on Security and Privacy (SP)*. 2017, pp. 3–18. DOI: 10.1109/SP.2017.41.
 - [70] Jeffrey Saltz, Michael Skirpan, Casey Fiesler, Micha Gorelick, Tom Yeh, Robert Heckman, Neil Dewar, and Nathan Beard. "Integrating Ethics within Machine Learning Courses". In: *ACM Trans. Comput. Educ.* 19.4 (2019). DOI: 10.1145/3341164.



Appendix

A.1 Comparing the baseline clustering models

Table A.1: Baseline clustering metrics and anomaly information for K-Means and GMM

Model	Silhouette	ARI	NMI	AIC	BIC
GMM	0.41	0.06	0.03	286328.97	286428.19
Component	Points	Anomalies	Anomaly Ratio		
0	42862	14862	0.35		
1	18203	10203	0.56		
Model	Silhouette	ARI	NMI	WSSSE	
K-Means	0.64	0.05	0.04	598238.23	
Component	Points	Anomalies	Anomaly Ratio		
0	54520	20520	0.38		
1	6545	4545	0.69		

A.2 Comparing the tuned clustering models

Table A.2: Tuned clustering metrics and anomaly information for K-Means and GMM

Model	Silhouette	ARI	NMI	AIC	BIC
GMM	0.65	0.18	0.15	179297.98	180100.73
Component	Points	Anomalies	Anomaly Ratio		
0	4645	1645	0.35		
1	4069	2168	0.53		
2	4075	2122	0.52		
3	21572	1572	0.07		
4	3362	2150	0.64		
5	3103	2105	0.68		
6	937	838	0.89		
7	6031	2031	0.34		
8	85	85	1.00		
9	18	18	1.00		
10	1408	1408	1.00		
11	1148	1148	1.00		
12	85	85	1.00		
13	719	672	0.93		
14	2357	1355	0.57		
15	2067	1279	0.62		
16	5366	4366	0.81		
17	18	18	1.00		
Model	Silhouette	ARI	NMI	WSSSE	
K-Means	0.73	0.11	0.07	28946.81	
Cluster	Points	Anomalies	Anomaly Ratio		
0	8403	3610	0.43		
1	4933	2934	0.59		
2	4939	2939	0.60		
3	25151	5151	0.20		
4	4455	3243	0.73		
5	4492	3285	0.73		
6	442	441	1.00		
7	8147	3359	0.41		
8	103	103	1.00		